ARCHITECTURE & IMPLEMENTATION PLAN

ECRH 日志系统架构与实施方案 V19

AUTHOR
FUSION DATA PLATFORM TEAM

DATE
May 09, 2026

BASELINE
V18 Design Baseline

Executive Summary

本文档提出了 ECRH 日志系统 V19 的架构与实施方案。作为进入 MVP（最小可行产品）阶段的完整冻结版，本版本在 V18 的设计基线之上，正式关闭了所有 P0/P1 级阻断项风险。重点更新包括：统一并简化了关系型数据库的迁移与快照语义、明确了基于确定性优先级的 shot 聚合与失败事实源规则、重构了事件防重与出站投递机制，并将跨炮聚合与重投影（Reprojection）组件前置到 MVP 范围。本系统专为支撑长脉冲实验的数据归档与事后分析而设计，确保在高并发吞吐及多类异常中断场景下维持严密的数据安全与全局一致性。

版本：V19(在 V18 设计基线上，关闭 P0/P1 阻断项，形成可进入 MVP 的完整冻结版)
日期:2026-05-09
适用对象:聚变装置 ECRH 系统 / 长脉冲实验 / 事后复盘 / 模拟数据驱动开发 / 真实 PLC 平滑切换

0.0 V18 → V19 变更摘要

类别	V18 → V19
评审结论	V18 方向正确，但 P0 设计风险未关闭；V19 作为完整 MVP 冻结版承接 V18 全文并内联修订
Migration ADR	不再混写 generated stored 目标态与 trigger 过渡态；MVP 目标态统一为普通事实列 + 视图 / 偏序索引
shot **聚合规则**	`data_completeness` 改为确定性优先级 + 冲突矩阵 + 8 状态测试矩阵
失败事实源	删除 `shots.failure_category / termination_reason` 双写摘要；以 `shot_failures` 为唯一失败事实源
失败分类	增加 `failure_layer`，补 `PARSER_FAILURE / UPLOAD_FAILURE / RECOVERY_FAILURE`
events **入库**	dedup 成功后才插入 `events` 并分配 `event_id`； `event_id` 不承诺连续
event_dedup_keys	删除无去重语义的 `event_type` 冗余列；同 key 不同 type 视为契约冲突并显式失败
Outbox	`idx_outbox_dispatch` 改按 `outbox_id` 排序； `outbox_id` 是 CH `_version` 唯一来源
CH preview	`ch_waveform_preview` 改 `ReplicatedReplacingMergeTree(..., _version)`，分区粒度改为 `intDiv(shot_no, 10000)`
reprojection	`reprojection_jobs` 前移到 MVP，支撑 CH events / preview 重投影与对账修复
运维闭环	明确 upload reaper 必须 abort multipart、HDF5 INSERT/UPDATE 都触发、prod-shadow dataset CHECK 落库
API **边界**	补齐 `valid_only=true` 两级窗口为空时的响应；大消息外置对象存储并显式 warning

0.1 对 V18 评审意见的判定

总体判定:评审意见主体合理，P0 列表中 6.5 项成立。V19 采纳核心风险判断，但不机械采纳所有实现建议。

编号	判定	V19 处理
P0-1 `data_completeness` 规则冲突	成立	改为确定性优先级、冲突矩阵、测试矩阵
P0-2 Migration ADR 生成列矛盾	成立	MVP 取消分区表 `GENERATED STORED` 目标态
P0-3 `shot_failures` 分区 FK 风险	成立	PG 16.4+ 保留 FK，但 14.0 必须 PoC
P0-4 `events` 先取全局序列	部分成立	dedup 成功后才插入 `events`；不承诺 `event_id` 连续
P0-5 CH preview `_version` 无效	成立	改 `ReplicatedReplacingMergeTree(..., _version)`
P0-6 `shots` 与 `shot_failures` 双写	成立	删除 `shots.failure_category / termination_reason`
P0-7 outbox dispatch 排序键错误	成立	dispatch 顺序改 `outbox_id`；`sequence_no` 废弃
P1-1 失败分类缺 pipeline/transport	成立	增加 `failure_layer`，并补 pipeline/transport 分类
P1-2 dedup hash 缺 `event_type`	部分成立	不把 `event_type` 入 hash；删冗余列，契约冲突显式报错
P1-3 `valid_only` 短炮兜底未定义	成立	补两级窗口都为空时的响应语义
P1-4 `outbox.idempotency_key` 无唯一语义	成立	新增 outbox idempotency ledger
P1-5 CH preview 分区过细	成立	分区改为 `intDiv(shot_no, 10000)`
P1-6 `reprojection_jobs` 被延后	成立	前移到 MVP 最小表
P1-7 upload reaper 未 abort multipart	成立	reaper 必须先调用 storage abort
P1-8 `time_sync_quality` 选行语义不明	成立	写入时按 measured_at 最近有效规则选行

编号	判定	V19 处理
P1-11 prod-shadow 仅靠前缀约定	成立	prod 与 prod-shadow 各自加 DB CHECK
P1-13 HDF5 mirror INSERT 漏触发	成立	INSERT / UPDATE 都触发入队

关键修正: 不采纳 `SELECT max(in_shot_seq)+1 ... FOR UPDATE` 作为 events 序列方案。它会把同 shot 并发事件写入串行化，且事务失败后仍可能留下诊断歧义。V19 明确 `event_id` 是 surrogate key，不是连续业务序号；事件排序基准为 `event_epoch_ns, ingestion_at, event_id`。

0.2 V19 MVP 冻结决策

1. PostgreSQL 目标版本为 **16.4+**，低于该版本不进入 MVP。
2. MVP 目标态不使用分区表上的 `GENERATED ALWAYS AS ... STORED`。
3. `duration_s`、`include_in_normal_stats` 不落表，由视图或查询谓词表达。
4. `shots` 只保存 shot 当前事实状态，不保存失败摘要字段。
5. `shot_failures` 是失败事实源；UI/API 走 `v_shots_with_latest_failure`。
6. `events` dedup 成功后才分配 `event_id`，dedup miss 不消耗编号。
7. `outbox_id` 是所有 CH `_version` 的唯一来源。
8. CH 只保存派生副本，在线查询不得依赖 `FINAL`。
9. `reprojection_jobs` 前移到 MVP。
10. prod / prod-shadow 必须有独立 dataset CHECK。
11. V19 不新增静默降级路径；所有异常契约冲突必须显式返回错误、写审计或进入 DLQ。

---

0. 设计前提

0.1 场景定位

维度	取值
核心目的	**事后复盘 / 数据归档 / 跨炮分析 / 物理研究**
实时性要求	**低**(实时保护与实时画板由 PLC/HMI/SCADA 自身硬件链路完成)
采集对象	TDMS 波形文件 + PLC/HMI 事件流
复盘窗口	**L4/L5 总览**:炮结束后 ≤ 30 min;**L0 首窗**:≤ 45 min;**L0 全量**:≤ 90 min
历史保留	Zarr 永久;TDMS 原始默认进入冷归档并保留 ≥ 5 年(MVP 热层按 180 天规划);事件 ≥ 10 年

0.2 数据规模(保留 V14 三档,含副本放大)

维度	当前 (MVP)	近期	远期最坏	远期典型
通道数	50	100	200	200
采样率	1 MHz	1 MHz	10 MHz	10 MHz / 1 MHz
单点字节	4 B	4 B	8 B	8 B / 4 B
持续吞吐	200 MB/s	400 MB/s	16 GB/s ⚠️	8.4 GB/s
单炮时长	1000 s	3000 s	10000 s	10000 s
单炮原始量	**200 GB**	**1.2 TB**	**160 TB**	**84 TB**
压缩比	3-4×	3-4×	2-3×	2-3×
单炮压缩后	50-70 GB	300-400 GB	60-80 TB	30-40 TB
炮频率	5-10/天	3-5/天	1/天	1-2/天

年化口径(沿用 V14)

```
active_experiment_days_per_year:
  - 90  天 : 保守
  - 180 天 : 采购基线(推荐)
  - 365 天 : 上界压力,不作为采购默认
```

MVP 第 1 年实占建议区间(把 raw TDMS 5 年保留折算进年度面容量):

规划口径	估算前提	第 1 年实占
保守	90 天	**0.35-0.75 PB**
基线(推荐)	180 天	**0.70-1.50 PB**
高负荷上界	365 天	**1.20-2.60 PB**

0.3 数据来源类型

```
TDMS 波形文件 : 真实(NI-DAQmx 持续采集)
PLC/HMI 事件   : 开发期模拟,生产期真实 PLC/HMI/SCADA/SOE 推送
```

四种 data_origin: `REAL` / `SIMULATED` / `SYNTHETIC` / `REPLAYED` 。**prod 仅允许 REAL,通过三层守卫强制 (API + DB trigger + prod-only CHECK)。**

---

1. 总体架构

1.1 架构图(沿用 V14)

```
═══════════════════════════════════════════════════════════════════════
[OT 网 / DAQ]
  PLC/HMI/SCADA/SOE  ──(mTLS, 客户端证书)──►  Edge Collector / Bastion
  DAQ 主机 (NI PXIe) ──(本地 NVMe RAID0, NI-DAQmx) ►  TDMS 分段文件
═══════════════════════════════════════════════════════════════════════
[DMZ / 单向化数据通道]
   - 控制面:mTLS + 防火墙白名单
   - 数据面:DAQ → MinIO 上传走专网 + 应用层鉴权;事件走 mTLS 推送
                                  │
                                  ▼
                     ┌──────────────────────┐
                     │ Ingest Gateway       │   OIDC + mTLS + schema 校验
                     │ (Envoy / Kong)       │   + Idempotency-Key + RateLimit
                     └──────────────────────┘
═══════════════════════│═══════════════│═══════════════════════════════
[App 网]              │               │
                      ▼               ▼
          ┌──────────────────┐  ┌──────────────────┐
          │ TDMS Uploader    │  │ Event Normalizer │
          │ (multipart 会话) │  │ (PG + outbox v2) │
          │ + upload_sessions│  │ + shot-scoped dedup │
          │ + reconcile      │  │                  │
          └──────────────────┘  └──────────────────┘
                  │                       │
                  ▼                       │
          ┌──────────────────────┐        │
          │ Object Storage       │        │
          │ MinIO (EC 4+2)       │        │
          │  ecrh-raw-tdms/      │        │
          │  ecrh-zarr/          │        │
          │  ecrh-evidence/      │  ◄── MVP 起 versioning + 不可删 policy
          │  ecrh-audit/         │  ◄── MVP 起 versioning + 不可删 policy
          │  ecrh-reports/       │        │
          └──────────────────────┘        │
                  │                       │
                  ▼                       │
          ┌──────────────────────────┐    │
          │ Parser Workflow Engine   │    │
          │ (Celery+PG lease;Temporal│    │
          │  二期)                   │    │
          │  - PG lease + fencing    │    │
          │  - per-segment 状态      │    │
          │  - checkpoint 恢复       │    │
          │  - immutable build       │    │
          │  - manifest activation   │    │
          │  - activation 重试可观测 │    │
          └──────────────────────────┘    │
                  │                       │
       ┌──────────┴──────────┬────────────┴──────────┐
       ▼                     ▼                       ▼
┌──────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│ PostgreSQL   │  │ ClickHouse       │  │ Redis (optional cache) │
│ (事实源)     │◄─┤ (分析副本)       │  │ MVP 默认不部署         │
│ + system_env │  │ ch_events_wide   │  │                        │
│ + prod CHECK │  │ ch_events_current │  │                        │
│ events       │  │   全字段 argMax  │  │                        │
│ outbox v2    │  │ ch_waveform_prev │  │                        │
│ parser_jobs  │  └──────────────────┘  └──────────────────────┘
│ object_reconcile_jobs ◄── 新增                  ▲
│ audit_log    │            │ Outbox Consumer Pool
└──────────────┘            │ + delivery token + replay tool
```

```
═══════╪═══════════════════╪═══════════════════════════════════════════
[控制室 / 用户网]
        ▼
┌──────────────────────────────────────────────────────────────────┐
│ API Gateway (OIDC + RBAC + Idempotency-Key + 限流)                 │
│  /events  /shots  /waveforms  /alarms  /protection  /files  /admin  │
│  /admin/reconcile-objects  /admin/prod-guard/self-test  /audit     │
│  /data-quality                                                     │
└──────────────────────────────────────────────────────────────────┘

═══════════════════════════════════════════════════════════════════════
[开发/集成环境特有]
  ecrh-simulator (仅 dev/staging)  ──►  /v1/internal/events:bulk-import
                                          (prod 部署不存在该路由 + 镜像 + service)
[prod-shadow 命名空间]
  真实 PLC 镜像流量 → prod-shadow → 对比 dashboard,不进 prod 事实库
  与 prod 事实库 0 网络通路,Network Policy + Firewall 双重隔离

═══════════════════════════════════════════════════════════════════════
[观测]    Prometheus + Grafana + Loki + Alertmanager + OpenTelemetry
[备份]    PG: pgBackRest PITR + 异地 / Object: 跨区复制 /
          CH: clickhouse-backup→S3 / Object Lock/WORM 在试运行前启用
═══════════════════════════════════════════════════════════════════════
```

1.2 13 条核心设计原则

```
 1. 复盘优先          : "炮后查得快、查得准、查得全"
 2. 单一路径          : 仅 Bulk Path,无 Live Path
 3. 事实源单一化      : PostgreSQL 是元数据/事件/业务事实唯一权威
 4. 大对象走文件      : TDMS 原始 + Zarr 全量波形只在对象存储
 5. 分析副本派生化    : ClickHouse 仅存预览点和聚合
 6. 缓存可重建        : Redis 一切可从 PG/CH/对象存储重建
 7. 反对应用层双写    : 派生层只通过 outbox 消费
 8. 多级降采样        : L0(1MHz) → L1 → L2 → L3 → L4 → L5(1Hz)
 9. 长脉冲友好        : Zarr chunk 按"目标字节"而非固定秒
10. 演进式扩展        : 每组件有明确激活触发条件(§28)
11. 工程化收敛        : MVP 只做闭环必需
12. 数据来源可溯源    : data_origin 贯穿入库/查询/导出/水印
13. 长任务可恢复      : Parser/Outbox 必须可 checkpoint、可 fencing、可重放
14. 失败语义分层化    : 实验炮失败 ≠ 上传失败 ≠ Parser 失败,不得混用单一状态代替
```

---

2. 数据通路设计

2.1 单路径采集

DAQ 写入

```
DAQ 主机
  ├─ NI-DAQmx 持续采集
  ├─ 写本地 NVMe RAID0 TDMS(每 100 s 自动 segment)
  ├─ shot_end → closeFile + 计算 sha256
  └─ Health 上报到 Prometheus push gateway
```

Shot 上传(multipart 会话化)

```
shot_end 触发:
  1. 关闭 TDMS segment + 生成 manifest.json
  2. POST /v1/files:upload-init
       → 第一步:INSERT INTO upload_sessions(status='INITIATED', storage_upload_id=NULL, sess
ion_expires_at, ...)
       → 第二步:调用对象存储 CreateMultipartUpload
       → 第三步:UPDATE upload_sessions SET storage_upload_id=..., status='UPLOADING'
       → 返回 upload_id + storage_upload_id + `upload_mode` + 预签名 URL 列表 + part 配置
       → part_size 动态:
           < 64 MB 文件 : single PUT 或 multipart 1 part
           ≤ 50 GB 文件 : 64 MB/part
           50-500 GB    : 256 MB/part
           > 500 GB     : 512 MB-1 GB/part
  3. `upload_mode=SINGLE_PUT` 时直接 PUT;`upload_mode=MULTIPART` 时并发上传(每 part PUT /v1/f
iles:upload-part 落 ETag)
  4. POST /v1/files:upload-complete
       → 校验所有 part ETag + parts manifest sha256 + 总 sha256
       → 写 data_files
       → 写 outbox v2: file.tdms.uploaded
  5. Parser 工作流被触发(MVP: Celery signal;二期可切 Temporal)
  6. DAQ 主机本地保留 7 天作为冗余
```

V16 补充: - `upload-complete` 写对象成功但 PG 事务失败时,**自动** INSERT `object_reconcile_jobs`(新增表,见 §4.2),由独立 reconciler 扫描、补偿或隔离 - `upload_sessions.reconcile_status` 字段记录孤儿状态: `NONE / ORPHAN_FOUND / REPAIRED / QUARANTINED` - init 任一步失败都必须有补偿:PG INITIATED 残留允许重试,对象存储 multipart 残留交给 lifecycle abort + reconcile 双兜底 - 超过 `session_expires_at` 的未完成会话由 reaper 回收 - 预签名 URL 必须绑定 `upload_id / object_key / part_no / content-length / checksum`

V19 扩展:失败炮 / 小文件 / 本地恢复 - `64 MB` 是 target `part_size` / target `chunk_size`，**不是**最小文件大小；严禁补零凑满，严禁把多个短炮合并成一个 Zarr 版本 - shot 是最小业务边界；短炮、失败炮、部分数据炮都必须保留 shot 级边界和独立 manifest - 受控中止路径:

```
operator abort / interlock / protection trip
  → DAQ 收到 abort marker
  → flush buffers
  → close TDMS segment
  → generate partial manifest
  → upload TDMS
  → parser best-effort
  → shots.status = ABORTED
  → shots.data_completeness = PARTIAL
  → create shot_failures row
```

- 非受控中断路径:

```
DAQ crash / power loss / OS crash
  → DAQ restart
  → daq-spool-scanner 扫描本地 NVMe spool
  → 找到 temp TDMS / unclosed TDMS / partial segment
  → tdms-recovery-worker 恢复可读前缀
  → recovery-manifest-generator 生成 recovery manifest
  → partial-upload-handler 上传 recovered object
  → parser best-effort
  → 可读: RECOVERED_PARTIAL
  → 不可读: CORRUPT / QUARANTINED
```

DAQ spool recovery 5 **组件设计边界**(V19 P1 **补齐**):

组件	输入	输出	幂等键	P1 SLO
`daq-spool-scanner`	本地 spool 目录、DAQ restart 事件、扫描 checkpoint	recovery candidate 清单	`host_id + path + inode + mtime`	restart 后 5 min 内完成首轮扫描
`tdms-recovery-worker`	candidate、原始文件句柄、恢复策略版本	readable prefix / recover fail	`file_sha256_prefix + tool_version`	单文件恢复 P95 ≤ 10 min
`recovery-manifest-generator`	恢复结果、通道探测结果	recovery manifest	`shot_no + source_file_id + recovery_attempt_no`	恢复成功后 2 min 内产出 manifest
`partial-upload-handler`	recovery manifest、recovered object、upload policy	`data_files` + upload session + outbox	`upload_id`	recovered object 上传成功率 ≥ 99%
`quarantine-review-tool`	quarantined object、parser/recovery 错误	处置决议 (RETRY/KEEP/DELETE)	`finding_id`	人工 review 不纳入在线 SLO

统一约束: - 五个组件都必须支持 checkpoint / resume - 任何重试都不得覆盖既有 recovery object，只能写新版本或新 attempt - 成功与失败都必须写审计和结构化指标

2.2 解析与金字塔

工作流引擎选型

- **MVP:** Celery + PG lease(不依赖 Redis)
- **触发引入 Temporal/Argo:** parser 平均时长 > 30 min 或需要取消/分支/补偿

Parser 工作流状态机

```
INIT → VERSION_ALLOCATED → SEGMENTS_PLANNED → CHANNELS_RUNNING (多协程)
     → MANIFEST_BUILT (写入 immutable build prefix)
     → VERIFIED (sha256 校验通过)
     → ACTIVATED (仅 PG 切换 active pointer / is_current)
     → COMPLETED

任意节点失败 → FAILED → 自动从 checkpoint 重启 ≤ 3 次
3 次失败 → 进入 quarantine 状态,告警人工介入
VERIFIED → ACTIVATED 单独追踪 activation_attempts / activation_error
VERIFIED → ACTIVATED 连续失败超过 5 次 → 自动 QUARANTINED + P1 告警
```

锁与 fencing

```database
UPDATE parser_jobs
SET lease_owner = $worker_id,
    lease_until = now() + INTERVAL '5 minutes',
    fencing_token = fencing_token + 1,
    last_heartbeat_at = now()
WHERE job_id = $job_id
  AND (lease_until IS NULL OR lease_until < now())
RETURNING fencing_token;
```

- worker 每 60s 续租;Zarr 写入路径 `v{n}/builds/fence-{token}/`,旧 token 无法 ACTIVATE
- 失去续租 → worker 立即停止写入并退出
- 独立 lease watchdog 每 30-60s 巡检 `last_heartbeat_at` 与 `lease_until`

流水

```
Parser Worker
  │
  ├─ 从 parser_jobs 领取(FOR UPDATE SKIP LOCKED) + 续租 lease
  ├─ 流式读 TDMS(npTDMS iter_chunks)
  ├─ 写 Zarr 到 immutable build prefix:v{n}/builds/fence-{T}/
  ├─ 流式构建 L0 → L5 金字塔(一遍扫描)
  ├─ 每完成 1 通道 → 更新 parser_job_files.status='DONE' + checkpoint_uri
  ├─ 全部完成 →
  │     - 校验 channel_sha256
  │     - 写 manifest.json + manifest_sha256
  │     - 写 waveform_channels / waveform_channel_gaps 元数据
  │     - 校验 manifest 一致
  │     - VERIFIED → ACTIVATED:UPDATE parser_jobs SET activation_attempts = activation_atte
mpts + 1
  │       在事务里:仅 UPDATE waveform_versions ACTIVE / is_current,
  │       旧版本置 SUPERSEDED,写 outbox;对象路径不 rename
  │     - ACTIVATE SQL 必须带 `fencing_token = :current_token` 条件
  │     - 失败 → 写 activation_error,整体回到 FAILED
  │
  └─ 失败 → 不删对象;置 FAILED;reaper 任务按 BUILDING 版本元数据过 24h 清理
```

Celery 边界: - Celery task 只负责领取和唤起 parser runner,不作为任务事实源 - PG `parser_jobs` / `parser_job_attempts` 才是唯一真相 - 必须覆盖 `kill -9 / worker 丢连接 / lease 过期 / duplicate dispatch / activate 重试` 五类 crash matrix

金字塔级别(沿用 V14)

级别	采样率	单炮 1000s 点数	字段	存储	主要用途
L0	1 MHz	10^9	value	Zarr	极致放大
L1	10 kHz	10^7	min/max/mean/rms	Zarr	高放大
L2	1 kHz	10^6	同上	Zarr	单炮全跨度
L3	100 Hz	10^5	同上	CH	UI 默认
L4	10 Hz	10^4	同上	CH	单炮总览
L5	1 Hz	10^3	同上	CH	跨炮聚合

Chunk 大小策略:

```python
TARGET_CHUNK_BYTES = 64 * 1024 * 1024  # 64 MB

def chunk_size_samples(sample_rate_hz: float, dtype_bytes: int) -> int:
    raw = TARGET_CHUNK_BYTES // dtype_bytes
    align = max(1, int(sample_rate_hz / 1000))
    return (raw // align) * align
```

采样率	字节	chunk 时长	字节/chunk
1 MHz × 4B	4 MB/s	16 s	64 MB
10 MHz × 8B	80 MB/s	0.8 s	64 MB

CH L2 策略: MVP 不把 L2 写入 CH。二期需通过单独 ADR、容量压测和 merge/mutation 预算评审。

V19 partial Zarr 规则: - Parser 必须支持 partial shot，而不是只接受完整炮 - 按实际样本数生成 Zarr array shape，不补零 - 缺失时间段不得写成 `0` - 小于目标 chunk 的通道允许只生成 1 个小 chunk - 每通道必须记录 `expected_samples` / `actual_samples` / `coverage_ratio` - 每个缺失窗口都必须写入 `waveform_channel_gaps` - TDMS 部分可读时生成 partial Zarr，并把 shot 标为 `PARTIAL` 或 `RECOVERED_PARTIAL` - TDMS 完全不可读时进入 `QUARANTINED`，不得生成 `ACTIVE` - `TRIP` / `INCIDENT` / `LEGAL_HOLD` 炮只要可读就必须生成 Zarr 并长期保留 - P0 至少保证 `expected_samples` / `actual_samples` 正确；`coverage_ratio` 列物化和专项统计可下放 P1

2.2A 失败炮 / 短脉冲炮 / 未完成炮业务模型(V19 P0 收敛)

V19 明确区分三类失败,不得再用一个 `ABORTED` 或一个 `FAILED` 覆盖全部语义:

类别	典型场景	属于什么失败
实验炮失败	联锁触发、保护跳闸、人工中止、水冷/真空/电源异常、只有几秒数据	物理实验 / 工控运行失败
上传失败	multipart 失败、对象已存在但 PG 无记录、upload session 过期	数据传输链路失败
Parser 失败	TDMS 解析失败、Zarr 写入失败、activation 失败、worker 崩溃	离线派生链路失败

shot 级状态从 V16 的单层 `status` 升级为双维模型:

```
shot_run_status:
  PLANNED
  RUNNING
  COMPLETED
  ABORTED
  INVALID

data_completeness:
  UNKNOWN
  COMPLETE
  PARTIAL
  RECOVERED_PARTIAL
  HEADER_ONLY
  NO_WAVEFORM
  CORRUPT
  MISSING
```

shot_run_status	data_completeness	含义
COMPLETED	COMPLETE	正常完整炮
COMPLETED	PARTIAL	实验完成,但数据缺失
ABORTED	PARTIAL	提前中止,但保留部分波形
ABORTED	RECOVERED_PARTIAL	非受控中断后恢复出部分可读数据
ABORTED	NO_WAVEFORM	只有事件,没有波形
INVALID	CORRUPT	文件损坏或不可恢复
INVALID	HEADER_ONLY	只有头信息,几乎无有效波形

`quality_status` 与 `data_completeness` 在 V19 中必须视为正交维度,不是替代关系:

轴	表达什么	典型值
`quality_status`	已落盘数据的物理量质量	`GOOD / DEGRADED / BAD / UNKNOWN`
`data_completeness`	数据存在性与完整性	`COMPLETE / PARTIAL / CORRUPT / NO_WAVEFORM ...`

示例：- `data_completeness=COMPLETE + quality_status=BAD`：数据齐全，但物理量质量差 - `data_completeness=PARTIAL + quality_status=GOOD`：已采到的那一段波形质量良好,只是整体不完整 - `data_completeness=CORRUPT + quality_status=UNKNOWN`：数据不可读,无法再给质量评级

正常性能统计默认按 V19 视图/谓词判断,不再落表保存生成列:

```
-- v_shots_compat.include_in_normal_stats =
(
  status = 'COMPLETED'
  AND data_completeness = 'COMPLETE'
  AND retention_class NOT IN ('TRIP','INCIDENT')
)
```

`TRIP / INCIDENT` 炮无论 `data_completeness` 如何,都不得进入 normal stats。

失败炮分析、保护事件分析、事故复盘视图则必须显式包含 `ABORTED / INVALID / PARTIAL / RECOVERED_PARTIAL / NO_WAVEFORM`。

shot 级 `data_completeness` 聚合规则(V19 P0 冻结):

```
输入:
  - 当前 shot 的 data_files
  - 当前 ACTIVE waveform_version 的 waveform_channels
  - parser / recovery finalize 结果

目标状态:
  UNKNOWN / COMPLETE / PARTIAL / RECOVERED_PARTIAL /
  HEADER_ONLY / NO_WAVEFORM / CORRUPT / MISSING

规则优先级:
  0. 若 shot 不预期产生波形,且仅有事件/保护记录 → NO_WAVEFORM
  1. 若预期有波形,但没有任何 source file、manifest、ACTIVE version → MISSING
  2. 若只有 header / manifest 可读,未产出任何有效通道样本 → HEADER_ONLY
  3. 若存在绑定当前 ACTIVE version 的 CORRUPT source_file,且无 replacement/recovery 证据 → CO
RRUPT
  4. 若当前 ACTIVE version 不存在,且仅有事件/保护记录 → NO_WAVEFORM
  5. 若存在 ACTIVE version,但 waveform_channels 为空或全部 completeness_status = 'NO_DATA' →
NO_WAVEFORM
  6. 若任一 channel.completeness_status = 'CORRUPT' → CORRUPT
  7. 若任一 channel.completeness_status = 'RECOVERED_PARTIAL' → RECOVERED_PARTIAL
  8. 若任一 channel.completeness_status IN ('PARTIAL','NO_DATA') → PARTIAL
  9. 若存在 coverage_ratio 且 min(coverage_ratio) < 0.95 → PARTIAL
 10. 若全部 channel.completeness_status = 'COMPLETE' 且 source_file 均可追溯 → COMPLETE
 11. 其余情况 → UNKNOWN,并产生 parser_quality finding
```

冲突矩阵:

输入组合	V19 结果	原因
source_file CORRUPT,无 ACTIVE version	CORRUPT	文件损坏且未恢复
source_file CORRUPT,ACTIVE version 来自 replacement 且全 COMPLETE	COMPLETE	当前可用事实优先,损坏文件作为审计证据保留
source_file CORRUPT,ACTIVE version 来自 recovery 且有通道缺口	RECOVERED_PARTIAL	恢复成功但不完整
channel CORRUPT 且 source_file COMPLETE	CORRUPT	当前通道不可读优先
同时存在 RECOVERED_PARTIAL 与 PARTIAL	RECOVERED_PARTIAL	恢复路径是更强语义
全部 channel NO_DATA	NO_WAVEFORM	不把全空通道伪装为 partial
无 ACTIVE version,仅有事件/保护记录	NO_WAVEFORM	与 MISSING 区分:没有文件缺失证据

实现约束: - shot 级聚合必须由 Parser finalize / Recovery finalize 单点写入,不得在多个服务重复推导 - 8 个目标状态每个至少 2 个输入组合单测;冲突矩阵每行必须有端到端测试 - `shots` 不再保存 `failure_category / termination_reason`;失败摘要由 `v_shots_with_latest_failure` 派生

time_sync_quality 传播规则(V19 P1 明确): - `time_sync_quality.confidence = LOW` 本身**不直接改变** `data_completeness` - 若 LOW 仅影响时间可信度但数据仍存在,则下调 `quality_status` 或在质量接口中标 `time_sync_degraded` - 若 LOW 导致无法可靠确定有效窗口边界,则 `valid_start_ns / valid_end_ns = NULL` - 只有"数据不存在 / 缺失 / 损坏 / 无法恢复"才改变 `data_completeness`

2.3 事件入库

流水

```
PLC/HMI/SCADA  或  ecrh-simulator
  │
  ├─ mTLS(生产)/ TLS+OIDC(开发)
  ├─ 客户端缓冲:本地 SQLite WAL(PG 不可用时缓冲 24h)
  └─ POST /v1/events:batch (≤1000) + Idempotency-Key 头
        │
        ▼
  Ingest Gateway
   ├─ schema 校验
   ├─ data_origin × env 校验(prod 仅 REAL)── 第 1 层
   ├─ 客户端身份注入 source_instance(来自 mTLS cert)
   │
  Event Normalizer
   ├─ 字典查表
   ├─ 计算 dedup_key_hash(公式见 §4.2)
   │
   ├─ BEGIN
   │  SELECT nextval('events_event_id_seq') AS event_id;
   │  -- shot-scoped dedup
   │  INSERT INTO event_dedup_keys(shot_no, dedup_key_hash, ..., first_event_id=event_id)
   │    ON CONFLICT (shot_no, dedup_key_hash) DO NOTHING
   │    RETURNING dedup_key_hash;
   │  IF 未冲突:
   │    INSERT INTO events(event_id=event_id, ...) ...
   │    INSERT INTO outbox ...
   │  COMMIT
   │
   ├─ DB trigger SECURITY DEFINER ── 第 2 层(读 system_environment)
   ├─ prod-only CHECK constraint ── 第 3 层(只在 prod 库存在)
   └─ 返回 envelope: {accepted, duplicated, rejected[], trace_id}
```

派生写入与事务边界

```
最小事务 = events 不可变写 + outbox 写
派生 alarm_instances 状态机改造:
  - 入库阶段:仅写 alarm_state_changes 流水(append-only)
  - 异步聚合任务:定时把流水卷算为当前 alarm_instances 状态
理由:避免单事务里同时持有 events 分区和 alarm_instances 行锁。
```

---

3. 存储分层

3.1 对象存储(MVP 安全前移)

```
ecrh-raw-tdms/
  保留:180 天 Hot/Warm → 5 年 Cold;MVP/试运行阶段不做自动删除。
  缩短保留前必须满足:
    1. 双 parser(A/B 实现)反演 sha256 一致
    2. 关键炮(`failure_category=PROTECTION_TRIP` 或 `retention_class IN ('KEY','TRIP','INCID
ENT','LEGAL_HOLD')`)永久保留
    3. 事故/复现申请期内永久保留
    4. 删除走 admin API + 双人审批 + audit_log

ecrh-zarr/
  保留:永久(自动分层 Hot 30d / Warm 2y / Cold ∞)
  年度任务:抽样反演 + sha256 对比;失败立即冻结 + 告警

ecrh-evidence/
  ⚠️ V15 修订:MVP 起就启用对象 versioning + 最小删除权限策略
  - bucket versioning = enabled
  - 服务账号无 DELETE 权限,仅有 PUT/GET
  - 普通业务角色禁止访问
  - 试运行前升级到 Object Lock(governance / compliance)

ecrh-audit/
  ⚠️ V15 修订:同 evidence
  - hash chain + bucket versioning(MVP)
  - 试运行前升级到 WORM

ecrh-reports/
```

3.2 Zarr V3 长期治理

```
- MVP 仍用 Zarr V3
- 每年发布"Zarr 健康报告":工具兼容性矩阵 / 元数据规范变更 / 抽样反演
- 一旦工具链断裂或 codec 失败:
    1. 触发 emergency parser run,把所有受影响版本重写
    2. 30 天内必须给出 ADR 决定迁移格式

⚠️ V15 修订:HDF5 镜像策略改为分级
- 关键炮(retention_class IN ('KEY','TRIP','INCIDENT','LEGAL_HOLD')) : 100% HDF5 镜像
- 普通炮 : 抽样 1% 镜像
- 关键炮判定后未生成镜像 → 自动加入 hdf5_mirror_jobs 队列
- 镜像与原 Zarr 通过 sha256 + numpy 反演双重验证
```

3.3 PG 容量

表	行数(年)	实际大小(含索引/JSONB)
events	5×10^7	80-150 GB
event_dedup_keys	5×10^7	30-40 GB
alarm_state_changes	10^6	2 GB
audit_log	5×10^6	10 GB
outbox / delivery_ledger	5×10^7	20-40 GB
object_reconcile_jobs	10^5	< 1 GB
其他	—	20-30 GB
总计		**180-270 GB/年**

3.4 ClickHouse 容量

```
ch_events_wide                  ~50 GB/年
ch_alarm_instances + protection ~5 GB/年
ch_waveform_preview L3          ~150 GB/年
ch_waveform_preview L4          ~15 GB/年
ch_waveform_preview L5          ~1.5 GB/年
总计:~220 GB/年(×2 副本 = ~440 GB/年)
```

3.5 Redis(简化)

- **MVP 默认不部署** Redis
- 先用 PG + 进程内缓存 + HTTP 缓存头跑通主流程
- 若引入 Redis:**不开 AOF**,只 RDB hourly + 完全可重建
- **不存** user session(移到 PG `user_sessions`)
- **候选用途**: 当前炮状态、parser 进度、字典缓存、查询结果缓存
- **缓存 key 必须含版本**: `ecrh:cache:wf:{shot_no}:{wv_id}:{channel}:{level}:{start}:{end}`
- Redis SLO **标注"仅启用** Redis **时生效",不作为** MVP **阻断项**

---

4. PostgreSQL 详细设计

4.1 表清单

类别	表名	MVP	备注
系统	**system_environment**	✅	**V15 新增**:不可绕过环境标识
主轴	shots	✅	RANGE(shot_no) 分区,含 `data_completeness`、`completion_ratio`、统计隔离字段
主轴	shot_failures	✅	**V17 新增**:失败原因实体表
时间质量	time_sync_quality	✅	
主数据	facilities	✅	**V16 新增**：最小 facility 表
主数据	daq_config_versions / plc_config_versions	✅	**V16 新增**：最小配置版本表
主数据	devices, signals	✅	signals 含 effective_from/to
主数据	signal_versions	✅	通道命名/单位变更历史
主数据	calibration_versions	✅	**V16 新增**：最小空表，先满足 FK 语义
字典	event_types / severities / sources / units	✅	
文件	data_files	✅	**V17 明确**:文件完整性 / close_state / recovery 元数据
文件	upload_sessions	✅	增 reconcile_status
文件	upload_parts	✅	

类别	表名	MVP	备注
文件	**object_reconcile_jobs**	✅	**V15 新增**:孤儿对象
文件	**object_reconcile_findings**	✅	**V16 新增**:可 checkpoint findings 子表
波形	waveform_versions	✅	增 reader_min_version / manifest_schema_version
波形	waveform_channels	✅	增 `coverage_ratio` / `completeness_status` / 有效窗口
波形	waveform_channel_gaps	✅	
事件	events(按 shot_no 分区)	✅	增 device_code_snapshot / signal_code_snapshot
事件	event_dedup_keys	✅	shot-scoped 幂等;hash 公式写死
事件	event_revisions	✅	默认 PENDING;增 approved_reason / approval_trace_id
报警	alarm_rules	✅	
报警	alarm_instances	✅	
报警	alarm_state_changes(月分区)	✅	
保护	protection_events / protection_event_causes	✅	
关联	event_waveform_windows	✅	
解析	parser_jobs	✅	增 activation_attempts / activation_error
解析	parser_job_files	✅	
解析	parser_job_attempts	✅	
同步	outbox(分区父表)	✅	RANGE(created_at) 月分区

类别	表名	MVP	备注
同步	outbox_dlq	✅	
同步	consumer_offsets	✅	
同步	consumer_delivery_ledger	✅	
同步	reprojection_jobs	✅	**V19 前移到 MVP**:CH events / preview 重投影与对账修复
审计	audit_log(月分区)	✅	
归档	hdf5_mirror_jobs	✅	**V16 新增**：关键炮镜像任务表
用户	users / roles / role_bindings / user_sessions	✅	

4.2 关键 DDL

说明(V19): - 本节 DDL 表达的是 **steady-state 目标模型** - brownfield `V16/V17/V18 → V19` 的在线迁移步骤以 §12.5 为准 - V19 MVP 不把分区表 generated stored 列作为目标态;若迁移中存在临时兼容列,必须在切换后移除

facilities / config versions / calibration_versions(最小占位表)

```
CREATE TABLE facilities (
  facility_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  facility_code  VARCHAR(32) NOT NULL UNIQUE,
  description    TEXT,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE daq_config_versions (
  daq_config_version_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  config_code           VARCHAR(64) NOT NULL UNIQUE,
  description           TEXT,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE plc_config_versions (
  plc_config_version_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  config_code           VARCHAR(64) NOT NULL UNIQUE,
  description           TEXT,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE calibration_versions (
  calibration_version_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  calibration_code       VARCHAR(64) NOT NULL UNIQUE,
  description            TEXT,
  created_at             TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

system_environment(V16 硬化,不可绕过环境标识)

```
CREATE TABLE system_environment (
  env             VARCHAR(16) NOT NULL CHECK (env IN ('dev','staging','prod-shadow','pro
d')),
  initialized_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  initialized_by  VARCHAR(64) NOT NULL,
  CONSTRAINT ck_singleton_row CHECK (env IS NOT NULL)
);
CREATE UNIQUE INDEX uk_system_environment_singleton ON system_environment ((true));

-- 部署时由特权角色一次性插入,业务角色无修改权限
REVOKE ALL ON system_environment FROM PUBLIC;
GRANT SELECT ON system_environment TO ecrh_app;
-- 仅 ecrh_guard_admin 角色可 INSERT/UPDATE/DELETE
```

SECURITY DEFINER trigger 函数(读 system_environment,业务角色不能改 env):

```
CREATE FUNCTION enforce_prod_real_data_origin()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog, ecrh
AS $$
DECLARE
  current_env VARCHAR(16);
BEGIN
  SELECT env INTO STRICT current_env FROM system_environment;
  IF current_env IN ('prod','prod-shadow') AND NEW.data_origin <> 'REAL' THEN
    RAISE EXCEPTION 'env=% rejects non-REAL data_origin=%', current_env, NEW.data_origin
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
EXCEPTION
  WHEN no_data_found THEN
    RAISE EXCEPTION 'system_environment not initialized; DB not ready for writes'
      USING ERRCODE = 'object_not_in_prerequisite_state';
END;
$$;

ALTER FUNCTION enforce_prod_real_data_origin() OWNER TO ecrh_guard_admin;
REVOKE EXECUTE ON FUNCTION enforce_prod_real_data_origin() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION enforce_prod_real_data_origin() TO ecrh_app;
```

fail-close 语义(V16 明确): - `system_environment` 缺失、空表、或读取异常时，**数据库拒绝业务写入** - `readiness` 必须在 `system_environment` 存在且 prod-only migration 校验通过后才返回绿色 - 备库 promote 流程必须先恢复/校验 `system_environment`，再开放写流量

prod-only CHECK 约束的部署与漂移规则(V16 新增):

```
1. prod-only CHECK 不再写在注释里；必须放在 migrations/prod-only/ 独立目录
2. Flyway prod baseline 缺失 prod-only migration 时，拒绝部署
3. /admin/prod-guard/self-test 必须额外执行“列出当前 prod CHECK 基线并比对”
4. 若约束被删除、NOT VALID、或名称不匹配，立即 P1 告警并阻断继续放量
```

shots

```sql
CREATE TABLE shots (
  shot_no                BIGINT NOT NULL,
  facility_id            BIGINT NOT NULL,
  status                 VARCHAR(16) NOT NULL CHECK (status IN
                           ('PLANNED','RUNNING','COMPLETED','ABORTED','INVALID')),
  data_completeness      VARCHAR(32) NOT NULL DEFAULT 'UNKNOWN' CHECK (data_completeness IN
                           ('UNKNOWN','COMPLETE','PARTIAL','RECOVERED_PARTIAL',
                            'HEADER_ONLY','NO_WAVEFORM','CORRUPT','MISSING')),
  shot_start_at          TIMESTAMPTZ,
  shot_end_at            TIMESTAMPTZ,
  shot_start_ns          BIGINT,
  shot_end_ns            BIGINT,
  expected_duration_ms   BIGINT,
  actual_duration_ms     BIGINT,
  completion_ratio       NUMERIC(8, 6),
  valid_start_ns         BIGINT,
  valid_end_ns           BIGINT,
  quality_status         VARCHAR(16) NOT NULL DEFAULT 'UNKNOWN',
  operator_id            BIGINT,
  campaign_id            BIGINT,
  -- V15 新增:配置版本溯源
  daq_config_version_id BIGINT,
  plc_config_version_id BIGINT,
  retention_class        VARCHAR(16) NOT NULL DEFAULT 'NORMAL'
                           CHECK (retention_class IN ('NORMAL','KEY','TRIP','INCIDENT','LEGA
L_HOLD')),
  experiment_purpose     TEXT,
  attributes             JSONB NOT NULL DEFAULT '{}'::jsonb
                           CHECK (octet_length(attributes::text) < 64*1024),
  data_origin            VARCHAR(16) NOT NULL DEFAULT 'REAL'
                           CHECK (data_origin IN ('REAL','SIMULATED','SYNTHETIC','REPLAYE
D')),
  dataset_id             VARCHAR(64),
  created_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (shot_no)
) PARTITION BY RANGE (shot_no);

CREATE INDEX idx_shots_origin ON shots(data_origin) WHERE data_origin != 'REAL';
CREATE INDEX idx_shots_retention ON shots(retention_class);
CREATE INDEX idx_shots_facility ON shots(facility_id) WHERE facility_id IS NOT NULL;
CREATE INDEX idx_shots_status_completeness ON shots(status, data_completeness);
CREATE INDEX idx_shots_normal_stats ON shots(shot_no)
  WHERE status = 'COMPLETED'
    AND data_completeness = 'COMPLETE'
    AND retention_class NOT IN ('TRIP','INCIDENT');
CREATE INDEX idx_shots_time_completeness ON shots(shot_start_at, data_completeness)
  WHERE NOT (
    status = 'COMPLETED'
    AND data_completeness = 'COMPLETE'
    AND retention_class NOT IN ('TRIP','INCIDENT')
  );
ALTER TABLE shots
  ADD CONSTRAINT fk_shots_facility FOREIGN KEY (facility_id) REFERENCES facilities(facility
_id);
ALTER TABLE shots
  ADD CONSTRAINT fk_shots_daq_cfg FOREIGN KEY (daq_config_version_id) REFERENCES daq_config
_versions(daq_config_version_id);
ALTER TABLE shots
  ADD CONSTRAINT fk_shots_plc_cfg FOREIGN KEY (plc_config_version_id) REFERENCES plc_config
_versions(plc_config_version_id);
ALTER TABLE shots
  ADD CONSTRAINT ck_shots_completion_ratio
  CHECK (completion_ratio IS NULL OR completion_ratio BETWEEN 0 AND 1);
```

```sql
ALTER TABLE shots
  ADD CONSTRAINT ck_shots_valid_range
  CHECK (
    valid_start_ns IS NULL
    OR valid_end_ns IS NULL
    OR valid_end_ns >= valid_start_ns
  );

CREATE TRIGGER trg_shots_prod_origin
BEFORE INSERT OR UPDATE ON shots
FOR EACH ROW EXECUTE FUNCTION enforce_prod_real_data_origin();

CREATE VIEW v_shots_compat AS
SELECT
  s.*,
  CASE
    WHEN s.actual_duration_ms IS NULL THEN NULL
    ELSE floor(s.actual_duration_ms / 1000.0)::INT
  END AS duration_s,
  (
    s.status = 'COMPLETED'
    AND s.data_completeness = 'COMPLETE'
    AND s.retention_class NOT IN ('TRIP','INCIDENT')
  ) AS include_in_normal_stats
FROM shots s;
```

默认规则(V19): - `actual_duration_ms` 是 shot 时长真值; `duration_s` 仅作为 `v_shots_compat` 兼容字段,不落表 - `include_in_normal_stats` 不落表,由视图 / 查询谓词 / 偏序索引表达,避免分区表 generated stored 迁移风险 - 若某个 campaign 需要特殊纳入规则,应通过视图或查询条件表达,不要回写 shot 行 - `completion_ratio = least(actual_duration_ms / expected_duration_ms, 1.0)`; 若 `expected_duration_ms IS NULL OR expected_duration_ms = 0`,则 `completion_ratio = NULL` - `shots` 不保存 `failure_category / termination_reason`;最新失败摘要只从 `shot_failures` 视图派生

shot_failures(V19 MVP 最小集)

```
CREATE TABLE shot_failures (
  failure_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no             BIGINT NOT NULL,
  facility_id         BIGINT NOT NULL REFERENCES facilities(facility_id),
  failure_time_at     TIMESTAMPTZ,
  failure_epoch_ns    BIGINT,
  failure_layer       VARCHAR(16) NOT NULL DEFAULT 'EXPERIMENT'
                      CHECK (failure_layer IN ('EXPERIMENT','TRANSPORT','PIPELINE')),
  failure_category    VARCHAR(64) NOT NULL CHECK (failure_category IN (
                        'INTERLOCK',
                        'PROTECTION_TRIP',
                        'OPERATOR_ABORT',
                        'POWER_SUPPLY',
                        'COOLING',
                        'VACUUM',
                        'MICROWAVE_SOURCE',
                        'MAGNET',
                        'COMMUNICATION',
                        'DAQ',
                        'SOFTWARE',
                        'STORAGE',
                        'NETWORK',
                        'PARSER_FAILURE',
                        'UPLOAD_FAILURE',
                        'RECOVERY_FAILURE',
                        'UNKNOWN'
                      )),
  termination_reason  VARCHAR(128),
  root_event_id       BIGINT,
  root_protection_event_id BIGINT,
  responsible_device_id BIGINT REFERENCES devices(device_id),
  evidence_uri        TEXT,
  severity            VARCHAR(16) NOT NULL DEFAULT 'MEDIUM'
                      CHECK (severity IN ('LOW','MEDIUM','HIGH','CRITICAL')),
  description         TEXT,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT fk_shot_failures_shot
    FOREIGN KEY (shot_no) REFERENCES shots(shot_no)
    DEFERRABLE INITIALLY IMMEDIATE
);

CREATE INDEX idx_shot_failures_shot ON shot_failures(shot_no);
CREATE INDEX idx_shot_failures_layer_category_time ON shot_failures(failure_layer, failure_
category, failure_time_at);
CREATE INDEX idx_shot_failures_time_facility ON shot_failures(facility_id, failure_time_a
t);
CREATE INDEX idx_shot_failures_root_event ON shot_failures(root_event_id)
  WHERE root_event_id IS NOT NULL;
CREATE INDEX idx_shot_failures_root_protection ON shot_failures(root_protection_event_id)
  WHERE root_protection_event_id IS NOT NULL;
```

partition FK 说明(V19): - 目标 PG 版本固定为 PostgreSQL 16.4+ - 保留 `shot_failures.shot_no → shots(shot_no)` 外键,但 14.0 前必须在近生产副本实测 FK 建立、批量插入、删除与回填代价 - 若目标规模下性能不可接受,允许切换为 `DEFERRABLE INITIALLY DEFERRED` 或表约束级触发器;禁止默认退化为纯应用层软引用

失败炮查询视图(V19)

```sql
CREATE VIEW v_shots_normal AS
SELECT *
FROM v_shots_compat
WHERE include_in_normal_stats = TRUE;

CREATE VIEW v_shots_with_latest_failure AS
SELECT s.*, sf.failure_id, sf.failure_layer, sf.failure_category,
       sf.termination_reason, sf.failure_time_at, sf.failure_epoch_ns,
       sf.severity AS failure_severity, sf.responsible_device_id,
       sf.root_protection_event_id, sf.evidence_uri
FROM shots s
LEFT JOIN LATERAL (
  SELECT *
  FROM shot_failures sf
  WHERE sf.shot_no = s.shot_no
  ORDER BY sf.failure_time_at DESC NULLS LAST, sf.created_at DESC, sf.failure_id DESC
  LIMIT 1
) sf ON TRUE;

CREATE VIEW v_shots_failures AS
SELECT *
FROM v_shots_with_latest_failure s
WHERE s.status IN ('ABORTED','INVALID')
   OR s.data_completeness IN ('PARTIAL','RECOVERED_PARTIAL','NO_WAVEFORM','CORRUPT','MISSIN
G','HEADER_ONLY')
   OR s.retention_class IN ('TRIP','INCIDENT');
```

events(增快照字段)

```
CREATE SEQUENCE events_event_id_seq CACHE 1000;

CREATE TABLE events (
  facility_id              BIGINT NOT NULL,
  shot_no                  BIGINT NOT NULL,
  event_id                 BIGINT NOT NULL DEFAULT nextval('events_event_id_seq'),
  event_time_at            TIMESTAMPTZ NOT NULL,
  event_epoch_ns           BIGINT NOT NULL,
  relative_time_ns         BIGINT,
  ingestion_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- 修订模型
  revision_no              INT  NOT NULL DEFAULT 1,
  is_retracted             BOOLEAN NOT NULL DEFAULT FALSE,
  correction_of_shot_no    BIGINT,
  correction_of_event_id   BIGINT,
  superseded_by_event_id   BIGINT,
  -- 来源
  source_system            VARCHAR(32) NOT NULL,
  source_instance          VARCHAR(64) NOT NULL,
  source_event_key         VARCHAR(128) NOT NULL,
  source_sequence_no       BIGINT,
  -- 业务(ID + code 快照)
  device_id                BIGINT NOT NULL,
  device_code_snapshot     VARCHAR(64) NOT NULL,
  signal_id                BIGINT,
  signal_code_snapshot     VARCHAR(64),
  event_family             VARCHAR(32) NOT NULL,
  event_type               VARCHAR(64) NOT NULL,
  severity                 VARCHAR(16) NOT NULL,
  quality                  VARCHAR(16) NOT NULL,
  message                  TEXT CHECK (message IS NULL OR octet_length(message) < 16*1024),
  correlation_id           UUID,
  dedup_key_hash           CHAR(64) NOT NULL,
  -- 值
  value_type               VARCHAR(16),
  bool_value               BOOLEAN,
  numeric_value            NUMERIC,
  enum_value               VARCHAR(64),
  string_value             TEXT,
  unit                     VARCHAR(16),
  -- 操作员
  actor_id                 BIGINT,
  actor_role               VARCHAR(32),
  actor_source             VARCHAR(32),
  client_ip                INET,
  -- 时间质量
  time_sync_quality_id     BIGINT,
  time_sync_match          VARCHAR(32),
  -- 来源溯源
  data_origin              VARCHAR(16) NOT NULL DEFAULT 'REAL'
                             CHECK (data_origin IN ('REAL','SIMULATED','SYNTHETIC','REPLAYE
D')),
  simulator_version        VARCHAR(32),
  simulation_seed          BIGINT,
  dataset_id               VARCHAR(64),
  ingest_batch_id          UUID,
  -- 原始
  raw_payload              JSONB
                             CHECK (raw_payload IS NULL OR octet_length(raw_payload::text) <
4*1024),
  raw_payload_uri          TEXT,
  raw_payload_truncated    BOOLEAN NOT NULL DEFAULT FALSE,
  schema_version           VARCHAR(8) NOT NULL DEFAULT 'v1',
```

```
  PRIMARY KEY (shot_no, event_id),
  UNIQUE (shot_no, dedup_key_hash)
) PARTITION BY RANGE (shot_no);

CREATE INDEX idx_events_signal_time ON events(signal_id, event_epoch_ns);
CREATE INDEX idx_events_family_severity_time ON events(event_family, severity, event_epoch_
ns);
CREATE INDEX idx_events_corr ON events(correlation_id) WHERE correlation_id IS NOT NULL;
CREATE INDEX idx_events_actor ON events(actor_id) WHERE actor_id IS NOT NULL;
CREATE INDEX idx_events_origin ON events(data_origin) WHERE data_origin != 'REAL';
CREATE INDEX idx_events_retracted ON events(shot_no) WHERE is_retracted = TRUE;
CREATE INDEX idx_events_shot_ingestion ON events(shot_no, ingestion_at);
ALTER TABLE events
  ADD CONSTRAINT fk_events_facility FOREIGN KEY (facility_id) REFERENCES facilities(facilit
y_id);

CREATE TRIGGER trg_events_prod_origin
BEFORE INSERT OR UPDATE ON events
FOR EACH ROW EXECUTE FUNCTION enforce_prod_real_data_origin();
```

V19 入库顺序:

```
BEGIN
  INSERT INTO event_dedup_keys(...)
    ON CONFLICT (shot_no, dedup_key_hash) DO NOTHING
    RETURNING shot_no, dedup_key_hash;

  IF no row returned:
    SELECT existing key fields;
    IF source fields conflict → raise DEDUP_CONTRACT_VIOLATION;
    ELSE return duplicated;

  INSERT INTO events(event_id DEFAULT, ...)
    RETURNING event_id;

  UPDATE event_dedup_keys SET first_event_id = :event_id
    WHERE shot_no = :shot_no AND dedup_key_hash = :hash;

  INSERT INTO outbox(..., idempotency_key, delivery_token, ...);
COMMIT
```

约束: - dedup 成功后才允许分配 `event_id` - `event_id` 是 surrogate key,不承诺同 shot 内连续 - 事件排序基准是 `event_epoch_ns, ingestion_at, event_id` - 同一 source key 对应不同 `event_type / signal_id / source_sequence_no` 必须显式报 `DEDUP_CONTRACT_VIOLATION`,不得静默吞掉

大消息外置规则 (V19): - 当 `events.message` 超过 16 KB,Normalizer 写完整内容到 `ecrh-evidence/messages/{shot_no}/{uuid}.json` - `events.message` 保存截断摘要, `raw_payload_truncated=true` , `raw_payload_uri` 指向完整对象 - 响应 envelope 显式返回 `warning: MESSAGE_TRUNCATED_TO_OBJECT` - 该流程是显式降级:必须写 audit 与 metrics,不得静默截断

快照字段填充规则(V19 沿用):

```
CREATE FUNCTION fill_event_snapshots()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, ecrh
AS $$
BEGIN
  SELECT device_code INTO NEW.device_code_snapshot
  FROM devices
  WHERE device_id = NEW.device_id;
  IF NEW.device_code_snapshot IS NULL THEN
    RAISE EXCEPTION 'device_id=% has no device_code snapshot', NEW.device_id;
  END IF;

  IF NEW.signal_id IS NOT NULL THEN
    SELECT signal_code INTO NEW.signal_code_snapshot
    FROM signals
    WHERE signal_id = NEW.signal_id;
  END IF;

  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_events_fill_snapshot
BEFORE INSERT ON events
FOR EACH ROW EXECUTE FUNCTION fill_event_snapshots();
```

event_dedup_keys(V19)

```
CREATE TABLE event_dedup_keys (
  shot_no           BIGINT NOT NULL,
  dedup_key_hash    CHAR(64) NOT NULL,
  dedup_scope       VARCHAR(16) NOT NULL DEFAULT 'SHOT'
                     CHECK (dedup_scope IN ('SHOT')),
  source_system     VARCHAR(32) NOT NULL,
  source_instance   VARCHAR(64) NOT NULL,
  source_event_key  VARCHAR(128) NOT NULL,
  source_sequence_no BIGINT,
  signal_id         BIGINT,
  first_seen_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  first_event_id    BIGINT,
  expires_at        TIMESTAMPTZ,
  PRIMARY KEY (shot_no, dedup_key_hash)
) PARTITION BY RANGE (shot_no);
CREATE INDEX idx_dedup_seen ON event_dedup_keys(first_seen_at);
```

dedup_key_hash 计算公式(V19):

```
dedup_key_hash = sha256_hex(
  shot_no                  || '⍰' ||
  source_system            || '⍰' ||
  source_instance          || '⍰' ||
  source_event_key         || '⍰' ||
  COALESCE(source_sequence_no::text, '∅')
)
```

语义说明:

场景	是否同 hash	处理
同 shot 内 PLC 重传同 source_event_key	是	幂等忽略
不同 shot 但 PLC source_event_key 重复	否	shot_no 不同导致 hash 不同,各自独立入库
历史回放(REPLAYED)同一 shot	否	dataset_id 不同 → 应走独立 shot 编号
同 shot 同 key 但 epoch_ns 漂移	是	仍视为同一逻辑消息重传
同 shot 同 key 但 signal/type 不同	是	显式报 DEDUP_CONTRACT_VIOLATION,暴露上游契约错误

REPLAYED / 测试约束: - REPLAYED 数据 dataset_id 必须以 `replay-` 前缀,且 shot_no 取自独立测试段(避免与真实 shot_no 冲突) - prod 永远不允许 REPLAYED

event_revisions(默认 PENDING + 增审计字段)

```
CREATE TABLE event_revisions (
  revision_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no              BIGINT NOT NULL,
  original_event_id    BIGINT NOT NULL,
  new_event_id         BIGINT NOT NULL,
  revision_no          INT NOT NULL,
  revision_type        VARCHAR(16) NOT NULL DEFAULT 'CORRECT'
                         CHECK (revision_type IN ('CORRECT','RETRACT','RESTORE')),
  reason               VARCHAR(64) NOT NULL,
  actor_id             BIGINT NOT NULL,
  approval_status      VARCHAR(16) NOT NULL DEFAULT 'PENDING'   -- ⚠️ V15:从 APPROVED 改为
PENDING
                         CHECK (approval_status IN ('PENDING','APPROVED','REJECTED')),
  approved_by          BIGINT,
  approved_reason      TEXT,                                    -- V15 新增
  approval_trace_id    UUID,                                    -- V15 新增
  applied_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  auto_expire_at       TIMESTAMPTZ GENERATED ALWAYS AS (applied_at + INTERVAL '30 days') ST
ORED,
  approved_at          TIMESTAMPTZ,
  before_payload       JSONB,
  after_payload        JSONB
);
CREATE INDEX idx_revisions_pending ON event_revisions(approval_status) WHERE approval_statu
s = 'PENDING';
```

审批生效规则(V16 明确): - `approval_status='PENDING'` 的修订**不得**推进 `events.superseded_by_event_id`、不得写 CH 派生 - 只有 `APPROVED` 后才允许生成新的当前态事件并写

outbox - 每日作业必须扫描 `auto_expire_at < now()` 且仍 `PENDING` 的修订，置为 `REJECTED/EXPIRED` 并告警

alarm_state_changes(最小 ISA-18.2 枚举前移到 MVP)

```sql
-- MVP 至少冻结状态枚举与合法值,异步聚合逻辑可在二期增强
CREATE TYPE alarm_lifecycle_state AS ENUM (
  'UNALM',      -- 正常
  'ALM',        -- 报警
  'ACK',        -- 已确认
  'RTNUN',      -- 恢复未确认
  'SHLV',       -- shelved
  'SUPPR',      -- suppressed
  'OOS'         -- out of service
);
```

upload_sessions / upload_parts(增 reconcile_status)

```
CREATE TABLE upload_sessions (
  upload_id               UUID PRIMARY KEY,
  storage_upload_id       TEXT,
  shot_no                 BIGINT NOT NULL,
  file_role               VARCHAR(32) NOT NULL,
  storage_bucket          VARCHAR(64) NOT NULL,
  object_key              TEXT NOT NULL,
  total_size_bytes        BIGINT NOT NULL,
  part_size_bytes         BIGINT NOT NULL,
  total_parts             INT NOT NULL,
  status                  VARCHAR(16) NOT NULL CHECK (status IN
                            ('INITIATED','UPLOADING','COMPLETING','COMPLETED','FAILED','ABO
RTED')),
  declared_sha256         CHAR(64) NOT NULL,
  actual_sha256           CHAR(64),
  completed_parts_sha256  CHAR(64),
  created_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
  session_expires_at      TIMESTAMPTZ NOT NULL,
  last_heartbeat_at       TIMESTAMPTZ,
  completed_part_count    INT NOT NULL DEFAULT 0,
  aborted_at              TIMESTAMPTZ,
  abort_reason            TEXT,
  completed_at            TIMESTAMPTZ,
  client_id               VARCHAR(64) NOT NULL,
  idempotency_key         VARCHAR(128),
  -- V15 新增
  reconcile_status        VARCHAR(16) NOT NULL DEFAULT 'NONE'
                            CHECK (reconcile_status IN ('NONE','ORPHAN_FOUND','REPAIRED','Q
UARANTINED')),
  reconcile_job_id        BIGINT,
  UNIQUE (client_id, idempotency_key)
);
CREATE UNIQUE INDEX uk_upload_storage_upload_id ON upload_sessions(storage_upload_id);
CREATE INDEX idx_upload_sessions_status ON upload_sessions(status, created_at);
CREATE INDEX idx_upload_sessions_expiry ON upload_sessions(session_expires_at)
  WHERE status IN ('INITIATED','UPLOADING','COMPLETING');
CREATE INDEX idx_upload_sessions_reconcile ON upload_sessions(reconcile_status)
  WHERE reconcile_status != 'NONE';
ALTER TABLE upload_sessions
  ADD CONSTRAINT ck_init_phases
  CHECK (
    (status = 'INITIATED' AND storage_upload_id IS NULL)
    OR
    (status IN ('UPLOADING','COMPLETING','COMPLETED','FAILED','ABORTED')
      AND storage_upload_id IS NOT NULL)
  );

-- V19 reaper 规则:
-- 1. 过期 upload_sessions 必须先调用对象存储 AbortMultipartUpload(storage_upload_id)
-- 2. storage abort 失败时记录 object_reconcile_jobs,等待 lifecycle 兜底;不得只更新 PG 状态
-- 3. reaper 每次处理必须写 audit / metrics,便于定位 multipart 残留

CREATE TABLE upload_parts (
  upload_id     UUID NOT NULL REFERENCES upload_sessions(upload_id) ON DELETE CASCADE,
  part_no       INT  NOT NULL,
  offset_bytes  BIGINT NOT NULL,
  size_bytes    INT  NOT NULL,
  etag          TEXT,
  sha256_hex    CHAR(64),
  status        VARCHAR(16) NOT NULL DEFAULT 'PENDING',
  uploaded_at   TIMESTAMPTZ,
  retry_count   INT NOT NULL DEFAULT 0,
```

```
  PRIMARY KEY (upload_id, part_no)
);
```

data_files(V19 明确文件完整性 / 恢复语义)

```
CREATE TABLE data_files (
  file_id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  facility_id            BIGINT NOT NULL REFERENCES facilities(facility_id),
  shot_no                BIGINT NOT NULL REFERENCES shots(shot_no),
  upload_id              UUID REFERENCES upload_sessions(upload_id),
  file_role              VARCHAR(32) NOT NULL CHECK (file_role IN
                           ('TDMS_SEGMENT','TDMS_MERGED','RECOVERY_TDMS','MANIFEST',
                            'RECOVERY_MANIFEST','EVIDENCE','HDF5_MIRROR')),
  storage_bucket         VARCHAR(64) NOT NULL,
  object_key             TEXT NOT NULL,
  object_version_id      TEXT,
  declared_sha256        CHAR(64) NOT NULL,
  actual_sha256          CHAR(64) NOT NULL,
  file_state             VARCHAR(32) NOT NULL DEFAULT 'COMPLETE' CHECK (file_state IN
                           ('COMPLETE','PARTIAL','RECOVERED_PARTIAL',
                            'CORRUPT','MISSING','QUARANTINED')),
  close_state            VARCHAR(32) NOT NULL DEFAULT 'CLOSED' CHECK (close_state IN
                           ('CLOSED','UNCLOSED','FORCED_CLOSED','RECOVERED','UNKNOWN')),
  expected_size_bytes    BIGINT,
  actual_size_bytes      BIGINT,
  valid_start_ns         BIGINT,
  valid_end_ns           BIGINT,
  recovery_tool_version  VARCHAR(64),
  recovery_note          TEXT,
  created_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  uploaded_at            TIMESTAMPTZ,
  CHECK (
    valid_start_ns IS NULL
    OR valid_end_ns IS NULL
    OR valid_end_ns >= valid_start_ns
  )
);

CREATE UNIQUE INDEX uk_data_files_object ON data_files(storage_bucket, object_key);
CREATE INDEX idx_data_files_shot ON data_files(shot_no, file_role);
CREATE INDEX idx_data_files_state ON data_files(file_state, close_state);
CREATE INDEX idx_data_files_shot_state ON data_files(shot_no, file_state);
```

`actual_size_bytes` **语义**(V19): - 对已上传对象,应尽量回填为对象存储返回的真实字节数 - 对 `MISSING` / `CORRUPT` 或仅完成头信息判定的恢复记录,允许为 `NULL`

object_reconcile_jobs(V15 新增)

```sql
CREATE TABLE object_reconcile_jobs (
  job_id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  scan_scope       VARCHAR(32) NOT NULL CHECK (scan_scope IN
                     ('UPLOAD_SESSION','BUCKET_PREFIX','SINGLE_OBJECT','PARSER_BUILD')),
  scope_args       JSONB NOT NULL,
  status           VARCHAR(16) NOT NULL DEFAULT 'PENDING' CHECK (status IN
                     ('PENDING','RUNNING','SUCCEEDED','FAILED','QUARANTINED')),
  -- 发现的孤儿对象明细
  orphan_count     INT NOT NULL DEFAULT 0,
  repaired_count   INT NOT NULL DEFAULT 0,
  quarantined_count INT NOT NULL DEFAULT 0,
  deleted_count    INT NOT NULL DEFAULT 0,
  -- 处置
  action_policy    VARCHAR(16) NOT NULL DEFAULT 'INVESTIGATE' CHECK (action_policy IN
                     ('INVESTIGATE','REPAIR','QUARANTINE','DELETE')),
  approved_by      BIGINT,
  approved_at      TIMESTAMPTZ,
  -- 元
  created_by       BIGINT,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at       TIMESTAMPTZ,
  completed_at     TIMESTAMPTZ,
  error_message    TEXT,
  trace_id         UUID
);
CREATE INDEX idx_orjobs_status ON object_reconcile_jobs(status, created_at);

CREATE TABLE object_reconcile_findings (
  finding_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  job_id           BIGINT NOT NULL REFERENCES object_reconcile_jobs(job_id) ON DELETE CASCA
DE,
  object_key       TEXT NOT NULL,
  storage_bucket   VARCHAR(64) NOT NULL,
  status           VARCHAR(16) NOT NULL DEFAULT 'PENDING'
                    CHECK (status IN ('PENDING','REPAIRED','QUARANTINED','DELETED','SKIPPE
D')),
  detected_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  processed_at     TIMESTAMPTZ,
  detail           JSONB
);
CREATE INDEX idx_orf_job_status ON object_reconcile_findings(job_id, status);
```

reconcile 流程:

```
1. 扫描器周期性触发:
   - 每 10 min 扫描 upload_sessions 中 status IN ('FAILED','ABORTED')
     且 storage_upload_id 在对象存储仍存在的 → 写 object_reconcile_jobs
   - 每 1 h 扫描 ecrh-zarr/ 中 BUILDING 版本超过 24h 的 → 同上
   - 每日全桶扫描 ecrh-raw-tdms/ 与 PG data_files 的差集
2. 状态机:
   PENDING → RUNNING → SUCCEEDED / FAILED / QUARANTINED
3. 处置策略:
   - REPAIR: 反向补 PG data_files 行(需 sha256 一致)
   - QUARANTINE: move 到 ecrh-quarantine/ 前缀,保留 30 天供人工调查
   - DELETE: 仅在双人审批后允许,写 audit_log
4. findings 明细:
   - 不再堆在单行 JSONB
   - 每个发现写 `object_reconcile_findings` 一行,支持逐条状态推进与断点恢复
5. 验收:人工注入 100 个孤儿对象,SUCCESS_RATE = 100%
```

parser_jobs(增 activation 字段)

```
CREATE TABLE parser_jobs (
  job_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no             BIGINT NOT NULL,
  parser_name         VARCHAR(64) NOT NULL,
  parser_version      VARCHAR(16) NOT NULL,
  priority            INT NOT NULL DEFAULT 100,
  status              VARCHAR(16) NOT NULL DEFAULT 'PENDING' CHECK (status IN
                        ('PENDING','RUNNING','SUCCEEDED','FAILED','CANCELLED','QUARANTINE
D')),
  -- lease + fencing
  lease_owner         VARCHAR(64),
  lease_until         TIMESTAMPTZ,
  fencing_token       BIGINT NOT NULL DEFAULT 0,
  last_heartbeat_at   TIMESTAMPTZ,
  lease_lost_at       TIMESTAMPTZ,
  runner_type         VARCHAR(16) NOT NULL DEFAULT 'CELERY'
                        CHECK (runner_type IN ('CELERY','TEMPORAL','MANUAL')),
  -- 取消
  cancel_requested    BOOLEAN NOT NULL DEFAULT FALSE,
  -- 重试
  attempts            INT NOT NULL DEFAULT 0,
  next_attempt_at     TIMESTAMPTZ,
  -- V15 新增:VERIFIED → ACTIVATED 单独追踪
  activation_attempts INT NOT NULL DEFAULT 0,
  max_activation_attempts INT NOT NULL DEFAULT 5,
  activation_error    TEXT,
  -- checkpoint
  checkpoint_uri      TEXT,
  -- 输出
  output_version_id   BIGINT,
  scheduled_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at          TIMESTAMPTZ,
  completed_at        TIMESTAMPTZ,
  error_message       TEXT,
  metrics             JSONB
);
CREATE INDEX idx_parser_jobs_pending
  ON parser_jobs(priority, scheduled_at) WHERE status = 'PENDING';
CREATE INDEX idx_parser_jobs_lease
  ON parser_jobs(lease_until) WHERE status = 'RUNNING';
CREATE INDEX idx_parser_jobs_watchdog
  ON parser_jobs(last_heartbeat_at, lease_until) WHERE status = 'RUNNING';
CREATE INDEX idx_parser_jobs_activation
  ON parser_jobs(activation_attempts) WHERE activation_attempts > 0;

CREATE FUNCTION check_activation_quarantine()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.activation_attempts >= NEW.max_activation_attempts
     AND OLD.status <> 'QUARANTINED' THEN
    NEW.status := 'QUARANTINED';
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_parser_jobs_activation_quarantine
BEFORE UPDATE ON parser_jobs
FOR EACH ROW EXECUTE FUNCTION check_activation_quarantine();

CREATE TABLE parser_job_files (
```

```sql
  job_id            BIGINT NOT NULL REFERENCES parser_jobs(job_id) ON DELETE CASCADE,
  file_id           BIGINT NOT NULL REFERENCES data_files(file_id),
  segment_start_s   INT,
  segment_end_s     INT,
  status            VARCHAR(16) NOT NULL DEFAULT 'PENDING' CHECK (status IN
                      ('PENDING','RUNNING','DONE','FAILED','SKIPPED')),
  checksum_verified BOOLEAN NOT NULL DEFAULT FALSE,
  bytes_processed   BIGINT NOT NULL DEFAULT 0,
  channels_done     INT NOT NULL DEFAULT 0,
  channels_total    INT,
  last_error        TEXT,
  PRIMARY KEY (job_id, file_id)
);

CREATE TABLE parser_job_attempts (
  attempt_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  job_id        BIGINT NOT NULL REFERENCES parser_jobs(job_id) ON DELETE CASCADE,
  attempt_no    INT NOT NULL,
  worker_id     VARCHAR(64) NOT NULL,
  fencing_token BIGINT NOT NULL,
  started_at    TIMESTAMPTZ NOT NULL,
  ended_at      TIMESTAMPTZ,
  outcome       VARCHAR(16) NOT NULL,
  error_message TEXT
);
```

waveform_versions(增 reader_min_version / manifest_schema_version)

```sql
CREATE TABLE waveform_versions (
  facility_id           BIGINT NOT NULL,
  waveform_version_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no              BIGINT NOT NULL,
  source_file_id       BIGINT NOT NULL REFERENCES data_files(file_id),
  version_no           INT NOT NULL,
  version_status       VARCHAR(16) NOT NULL CHECK (version_status IN
                        ('BUILDING','VALIDATING','ACTIVE','SUPERSEDED','FAILED','QUARANTINE
D')),
  is_current           BOOLEAN NOT NULL DEFAULT FALSE,
  storage_format       VARCHAR(16) NOT NULL DEFAULT 'ZARR_V3',
  storage_base_uri     TEXT NOT NULL,
  manifest_uri         TEXT NOT NULL,
  fencing_token        BIGINT NOT NULL,
  parser_name          VARCHAR(64) NOT NULL,
  parser_version       VARCHAR(16) NOT NULL,
  parser_args          JSONB,
  zarr_spec_version    VARCHAR(8) NOT NULL DEFAULT 'v3',
  codec                VARCHAR(32),
  compressor           VARCHAR(32),
  calibration_version_id BIGINT,                                -- nullable,二期补表
  timebase_id          BIGINT,
  -- V15 新增
  reader_min_version   VARCHAR(32)
                        CHECK (reader_min_version IS NULL OR reader_min_version ~ '^[a-z0-9_
-]+/[0-9]+\\.[0-9]+$'),
  manifest_schema_version VARCHAR(16) NOT NULL DEFAULT 'v1',    -- manifest 自身 schema 版本
  source_sha256        CHAR(64) NOT NULL,
  manifest_sha256      CHAR(64),
  total_bytes          BIGINT,
  pyramid_levels       VARCHAR(32) NOT NULL DEFAULT 'L0,L1,L2,L3,L4,L5',
  channel_count        INT,
  total_samples        BIGINT,
  shot_duration_s      INT,
  parse_duration_s     INT,
  activated_at         TIMESTAMPTZ,
  error_message        TEXT,
  created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at         TIMESTAMPTZ
);
CREATE UNIQUE INDEX uk_wv_shot_version ON waveform_versions(shot_no, version_no);
CREATE UNIQUE INDEX uk_wv_current ON waveform_versions(shot_no)
  WHERE is_current = TRUE AND version_status = 'ACTIVE';
ALTER TABLE waveform_versions
  ADD CONSTRAINT fk_wv_facility FOREIGN KEY (facility_id) REFERENCES facilities(facility_i
d);

CREATE TABLE waveform_channels (
  waveform_version_id BIGINT NOT NULL REFERENCES waveform_versions(waveform_version_id) ON
DELETE CASCADE,
  channel_id           BIGINT NOT NULL,
  channel_code         VARCHAR(64) NOT NULL,
  sample_rate_hz       NUMERIC(16, 3) NOT NULL,
  sample_dtype         VARCHAR(16) NOT NULL,
  level_bitmap         VARCHAR(32) NOT NULL DEFAULT 'L0,L1,L2,L3,L4,L5',
  codec                VARCHAR(32),
  compressor           VARCHAR(32),
  calibration_version_id BIGINT,
  timebase_id          BIGINT,
  chunk_target_bytes   BIGINT NOT NULL,
  chunk_count          INT NOT NULL,
  expected_samples     BIGINT NOT NULL,
  actual_samples       BIGINT NOT NULL,
```

```sql
  coverage_ratio      NUMERIC(8, 6),
  valid_start_ns      BIGINT,
  valid_end_ns        BIGINT,
  gap_count           INT NOT NULL DEFAULT 0,
  gap_samples         BIGINT NOT NULL DEFAULT 0,
  dropouts_count      INT NOT NULL DEFAULT 0,
  first_gap_ns        BIGINT,
  last_gap_ns         BIGINT,
  completeness_status VARCHAR(32) NOT NULL DEFAULT 'UNKNOWN' CHECK (completeness_status IN
                       ('UNKNOWN','COMPLETE','PARTIAL','RECOVERED_PARTIAL','NO_DATA','CORRU
PT')),
  quality_status      VARCHAR(16) NOT NULL DEFAULT 'GOOD' CHECK (quality_status IN
                       ('GOOD','DEGRADED','BAD','UNKNOWN')),
  quality_flags       JSONB NOT NULL DEFAULT '[]'::jsonb,
  stats_json          JSONB,
  PRIMARY KEY (waveform_version_id, channel_id)
);

CREATE TABLE waveform_channel_gaps (
  waveform_version_id BIGINT NOT NULL,
  channel_id          BIGINT NOT NULL,
  gap_no              INT NOT NULL,
  gap_start_ns        BIGINT NOT NULL,
  gap_end_ns          BIGINT NOT NULL,
  missing_samples     BIGINT NOT NULL,
  gap_reason          VARCHAR(32) NOT NULL DEFAULT 'UNKNOWN'
                      CHECK (gap_reason IN
                        ('DROP_OUT','NOT_RECORDED','TIMEBASE_GAP','RECOVERED_GAP','UNKNOW
N')),
  PRIMARY KEY (waveform_version_id, channel_id, gap_no),
  FOREIGN KEY (waveform_version_id, channel_id)
    REFERENCES waveform_channels(waveform_version_id, channel_id) ON DELETE CASCADE
);

ALTER TABLE parser_jobs
  ADD CONSTRAINT fk_parser_jobs_output_version
  FOREIGN KEY (output_version_id) REFERENCES waveform_versions(waveform_version_id);

ALTER TABLE waveform_channels
  ADD CONSTRAINT ck_waveform_channels_coverage_ratio
  CHECK (coverage_ratio IS NULL OR coverage_ratio BETWEEN 0 AND 1);

CREATE INDEX idx_waveform_channels_completeness
  ON waveform_channels(waveform_version_id, completeness_status);
CREATE INDEX idx_waveform_channels_coverage
  ON waveform_channels(waveform_version_id, coverage_ratio);
```

coverage_ratio 规则 (V19): - `coverage_ratio = actual_samples / expected_samples` - 若 `expected_samples = 0`，则 `coverage_ratio = NULL`，不得伪造为 `0` 或 `1`

```
CREATE TABLE hdf5_mirror_jobs (
  mirror_job_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no            BIGINT NOT NULL,
  facility_id        BIGINT NOT NULL REFERENCES facilities(facility_id),
  waveform_version_id BIGINT REFERENCES waveform_versions(waveform_version_id),
  trigger_reason     VARCHAR(16) NOT NULL CHECK (trigger_reason IN ('RETENTION_CLASS','ANNU
AL_SAMPLE','MANUAL')),
  status             VARCHAR(16) NOT NULL DEFAULT 'PENDING'
                      CHECK (status IN ('PENDING','RUNNING','SUCCEEDED','FAILED','QUARANTIN
ED')),
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at       TIMESTAMPTZ,
  output_uri         TEXT,
  error_message      TEXT
);

CREATE FUNCTION enqueue_hdf5_mirror_on_retention_upgrade()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.retention_class IN ('KEY','TRIP','INCIDENT','LEGAL_HOLD')
     AND COALESCE(OLD.retention_class, 'NORMAL') <> NEW.retention_class THEN
    INSERT INTO hdf5_mirror_jobs(shot_no, facility_id, trigger_reason)
    VALUES (NEW.shot_no, NEW.facility_id, 'RETENTION_CLASS');
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_shots_enqueue_hdf5_mirror_insert
AFTER INSERT ON shots
FOR EACH ROW EXECUTE FUNCTION enqueue_hdf5_mirror_on_retention_upgrade();

CREATE TRIGGER trg_shots_enqueue_hdf5_mirror_update
AFTER UPDATE OF retention_class ON shots
FOR EACH ROW EXECUTE FUNCTION enqueue_hdf5_mirror_on_retention_upgrade();
```

time_sync_quality(沿用 V14)

```
CREATE TABLE time_sync_quality (
  time_sync_quality_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shot_no              BIGINT NOT NULL,
  source_system        VARCHAR(32) NOT NULL,
  source_instance      VARCHAR(64) NOT NULL,
  sync_protocol        VARCHAR(16) NOT NULL CHECK (sync_protocol IN
                         ('PTP','NTP','GPS','LOCAL','UNKNOWN')),
  sync_state           VARCHAR(16) NOT NULL CHECK (sync_state IN
                         ('LOCKED','HOLDOVER','FREE_RUNNING','UNSYNCED','UNKNOWN')),
  precision_ns         BIGINT,
  offset_ns            BIGINT,
  jitter_ns            BIGINT,
  last_sync_at         TIMESTAMPTZ,
  measured_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  confidence           VARCHAR(8) NOT NULL DEFAULT 'UNKNOWN' CHECK (confidence IN
                         ('HIGH','MEDIUM','LOW','UNKNOWN')),
  raw_metrics          JSONB
);
CREATE INDEX idx_tsq_shot_source ON time_sync_quality(shot_no, source_system, source_instan
ce);
```

time_sync 与 shot 质量关系(V19): - `time_sync_quality` 是独立质量轴,不等价于 `data_completeness` - `confidence='LOW'` 默认不把 shot 自动改写成 `PARTIAL` - 只有当时钟问题导致无法确定有效时间窗时,才把 `valid_*_ns` 置空并在质量接口标记 `time_sync_degraded` - events 写入时选择同 `(shot_no, source_system, source_instance)` 下 `measured_at <= event_time_at` 且最近的一行;若不存在,取 `measured_at` 最近的一行并暴露 `time_sync_match=NEAREST_AFTER_OR_UNKNOWN`

outbox v2(V19:以 outbox_id 统一 version 与派发顺序)

```
-- 父表(分区)
CREATE TABLE outbox (
  outbox_id        BIGINT GENERATED ALWAYS AS IDENTITY,
  aggregate_type   VARCHAR(32) NOT NULL,
  aggregate_id     BIGINT NOT NULL,
  sequence_no      BIGINT,    -- 迁移兼容字段;V19 起不得作为 CH _version 或 dispatch order
  event_type       VARCHAR(64) NOT NULL,
  destination      VARCHAR(32) NOT NULL,
  payload          JSONB NOT NULL CHECK (octet_length(payload::text) < 256*1024),
  status           VARCHAR(16) NOT NULL DEFAULT 'PENDING' CHECK (status IN
                     ('PENDING','IN_PROGRESS','PUBLISHED','FAILED','DLQ')),
  locked_by        VARCHAR(64),
  locked_until     TIMESTAMPTZ,
  retry_count      INT NOT NULL DEFAULT 0,
  next_attempt_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_error       TEXT,
  idempotency_key  VARCHAR(128) NOT NULL,
  delivery_token   VARCHAR(128) NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at     TIMESTAMPTZ,
  PRIMARY KEY (outbox_id, created_at)
) PARTITION BY RANGE (created_at);

CREATE UNIQUE INDEX uk_outbox_seq
  ON outbox(aggregate_type, aggregate_id, sequence_no, destination, created_at);
CREATE UNIQUE INDEX uk_outbox_delivery
  ON outbox(destination, delivery_token, created_at);

CREATE INDEX idx_outbox_dispatch
  ON outbox(destination, status, next_attempt_at, outbox_id)
  WHERE status IN ('PENDING','IN_PROGRESS');
CREATE INDEX idx_outbox_published
  ON outbox(status, published_at) WHERE status = 'PUBLISHED';

CREATE TABLE outbox_idempotency_keys (
  idempotency_key VARCHAR(128) PRIMARY KEY,
  outbox_id BIGINT NOT NULL,
  outbox_created_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  FOREIGN KEY (outbox_id, outbox_created_at) REFERENCES outbox(outbox_id, created_at)
);

COMMENT ON COLUMN outbox.sequence_no IS
  'Deprecated in V19. Never use as CH _version or dispatch order.';

-- 月分区模板(由 pg_partman 或自研脚本预创建未来 6 个月)
CREATE TABLE outbox_2026_05 PARTITION OF outbox
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
CREATE TABLE outbox_2026_06 PARTITION OF outbox
  FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
-- ... 自动创建到 2026-11

-- 自动建分区任务(每月 1 日运行;V19 P2 必须 ADR 化 pg_cron/pg_partman/调度器选择)
CREATE OR REPLACE FUNCTION ensure_outbox_partitions(months_ahead INT DEFAULT 6)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
  i INT;
  start_date DATE;
  end_date DATE;
  partition_name TEXT;
BEGIN
  FOR i IN 0..months_ahead LOOP
    start_date := date_trunc('month', now() + (i || ' months')::interval)::date;
```

```
    end_date := start_date + interval '1 month';
    partition_name := 'outbox_' || to_char(start_date, 'YYYY_MM');
    EXECUTE format(
      'CREATE TABLE IF NOT EXISTS %I PARTITION OF outbox FOR VALUES FROM (%L) TO (%L)',
      partition_name, start_date, end_date
    );
  END LOOP;
END $$;

-- 归档:PUBLISHED 30-90 天分区由归档任务 DETACH 后转 Parquet 落对象存储
-- V19 规则:
-- 1. Consumer 拉取与排序一律按 outbox_id
-- 2. CH _version 一律取 outbox_id
-- 3. sequence_no 仅用于迁移排障,禁止进入当前态决策
-- 4. outbox 写入必须同事务插入 outbox_idempotency_keys;重复 idempotency_key 必须显式失败或返回
既有 outbox_id
-- 5. delivery_token 不允许跨日重用;若跨分区重现,consumer 必须拒绝并写 DLQ

CREATE TABLE outbox_dlq (
  outbox_id        BIGINT NOT NULL,
  aggregate_type   VARCHAR(32) NOT NULL,
  aggregate_id     BIGINT NOT NULL,
  destination      VARCHAR(32) NOT NULL,
  payload          JSONB NOT NULL CHECK (octet_length(payload::text) < 256*1024),
  retry_count      INT NOT NULL,
  last_error       TEXT,
  moved_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolution_status VARCHAR(16) NOT NULL DEFAULT 'OPEN'
                     CHECK (resolution_status IN ('OPEN','ACKED','REPLAYED','IGNORED')),
  resolved_by      BIGINT,
  resolved_at      TIMESTAMPTZ,
  PRIMARY KEY (outbox_id)
);

CREATE TABLE consumer_offsets (
  consumer_name    VARCHAR(64) PRIMARY KEY,
  destination      VARCHAR(32) NOT NULL,
  last_outbox_id   BIGINT NOT NULL DEFAULT 0,
  last_seq_per_aggregate JSONB NOT NULL DEFAULT '{}'::jsonb,
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE consumer_delivery_ledger (
  consumer_name    VARCHAR(64) NOT NULL,
  destination      VARCHAR(32) NOT NULL,
  delivery_token   VARCHAR(128) NOT NULL,
  first_outbox_id  BIGINT NOT NULL,
  last_outbox_id   BIGINT NOT NULL,
  sink_ack_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (consumer_name, destination, delivery_token, sink_ack_at)
) PARTITION BY RANGE (sink_ack_at);

-- 同样按月分区,90 天后归档
```

reprojection_jobs(V19 MVP)

```
CREATE TABLE reprojection_jobs (
  job_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  scope VARCHAR(32) NOT NULL CHECK (scope IN ('SHOT','EVENT','WAVEFORM_VERSION','TIME_RANG
E')),
  target_destination VARCHAR(32) NOT NULL CHECK (target_destination IN ('CH_EVENTS','CH_PRE
VIEW','ALL')),
  scope_args JSONB NOT NULL CHECK (octet_length(scope_args::text) < 64*1024),
  status VARCHAR(16) NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING','RUNNING','SUCCEEDED','FAILED','QUARANTINED')),
  attempts INT NOT NULL DEFAULT 0,
  fencing_token UUID NOT NULL,
  last_error TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ
);
CREATE INDEX idx_reprojection_dispatch ON reprojection_jobs(status, created_at)
WHERE status IN ('PENDING','FAILED');
```

V19 规则: - PG 事实源修订、CH 当前态误差、CH preview 重生成均通过 reprojection_jobs 入队 - worker 必须持有 fencing_token;过期 token 写入一律拒绝 - 失败不得静默吞掉;重试耗尽进入 QUARANTINED 并 P1 告警

audit_log(MVP 起 versioning + 不可删)

```
ALTER TABLE audit_log
  ADD COLUMN facility_id BIGINT NOT NULL REFERENCES facilities(facility_id),
  ADD COLUMN prev_hash CHAR(64),
  ADD COLUMN this_hash CHAR(64);

-- MVP 阶段 ecrh-audit/ bucket 启用 versioning + lifecycle 阻止删除
-- 试运行前升级到 Object Lock(governance/compliance)
```

4.3 分区策略

表	分区维度	粒度	老化策略
shots	RANGE(shot_no)	1000 炮	永不删
events	RANGE(shot_no)	1000 炮	老分区 DETACH → S3 Parquet
event_dedup_keys	RANGE(shot_no)	1000 炮	同 events
event_waveform_windows	RANGE(shot_no)	1000 炮	同上
alarm_state_changes	RANGE(occurred_at)	月	> 5 年 DETACH
outbox	RANGE(created_at)	月	PUBLISHED 30-90 天归档;FAILED/DLQ 处置完成前保留
consumer_delivery_ledger	RANGE(sink_ack_at)	月	> 90 天归档
audit_log	RANGE(occurred_at)	月	> 7 年 DETACH

5. ClickHouse 详细设计

5.1 ch_events_wide

```
CREATE TABLE ch_events_wide (
  facility_id      UInt64,
  shot_no          UInt64,
  event_id         UInt64,
  revision_no      UInt32,
  event_time       DateTime64(6, 'UTC'),
  event_epoch_ns   Int64,
  relative_time_ns Int64,
  ingestion_at     DateTime64(3, 'UTC'),
  source_system    LowCardinality(String),
  source_instance  LowCardinality(String),
  device_id        UInt64,
  device_code      LowCardinality(String),
  signal_id        Nullable(UInt64),
  signal_code      String,
  event_family     LowCardinality(String),
  event_type       LowCardinality(String),
  severity         LowCardinality(String),
  quality          LowCardinality(String),
  message          String,
  correlation_id   UUID,
  value_type       LowCardinality(String),
  bool_value       Nullable(UInt8),
  numeric_value    Nullable(Float64),
  enum_value       Nullable(String),
  unit             LowCardinality(String),
  actor_id         Nullable(UInt64),
  actor_role       LowCardinality(String),
  data_origin      LowCardinality(String),
  dataset_id       LowCardinality(String),
  is_retracted     UInt8 DEFAULT 0,
  correction_of_event_id Nullable(UInt64),
  superseded_by_event_id Nullable(UInt64),
  time_sync_quality_id   Nullable(UInt64),
  schema_version   LowCardinality(String),
  _version         UInt64,
  _delivery_token  String
)
ENGINE = ReplicatedReplacingMergeTree(
  '/clickhouse/tables/{shard}/events_wide', '{replica}', _version)
PARTITION BY intDiv(shot_no, 1000)
ORDER BY (shot_no, event_id)
TTL event_time + INTERVAL 10 YEAR;
```

当前态视图(V16:全字段 argMax):

```sql
CREATE VIEW ch_events_current_v AS
SELECT
  facility_id,
  shot_no,
  event_id,
  argMax(revision_no,             _version) AS revision_no,
  argMax(event_time,              _version) AS event_time,
  argMax(event_epoch_ns,          _version) AS event_epoch_ns,
  argMax(relative_time_ns,        _version) AS relative_time_ns,
  argMax(ingestion_at,            _version) AS ingestion_at,
  argMax(source_system,           _version) AS source_system,
  argMax(source_instance,         _version) AS source_instance,
  argMax(device_id,               _version) AS device_id,
  argMax(device_code,             _version) AS device_code,
  argMax(signal_id,               _version) AS signal_id,
  argMax(signal_code,             _version) AS signal_code,
  argMax(event_family,            _version) AS event_family,
  argMax(event_type,              _version) AS event_type,
  argMax(severity,                _version) AS severity,
  argMax(quality,                 _version) AS quality,
  argMax(message,                 _version) AS message,
  argMax(correlation_id,          _version) AS correlation_id,
  argMax(value_type,              _version) AS value_type,
  argMax(bool_value,              _version) AS bool_value,
  argMax(numeric_value,           _version) AS numeric_value,
  argMax(enum_value,              _version) AS enum_value,
  argMax(unit,                    _version) AS unit,
  argMax(actor_id,                _version) AS actor_id,
  argMax(actor_role,              _version) AS actor_role,
  argMax(data_origin,             _version) AS data_origin,
  argMax(dataset_id,              _version) AS dataset_id,
  argMax(is_retracted,            _version) AS is_retracted,
  argMax(correction_of_event_id,  _version) AS correction_of_event_id,
  argMax(superseded_by_event_id,  _version) AS superseded_by_event_id,
  argMax(time_sync_quality_id,    _version) AS time_sync_quality_id,
  argMax(schema_version,          _version) AS schema_version,
  max(_version) AS current_version
FROM ch_events_wide
GROUP BY facility_id, shot_no, event_id;
```

性能预算: - 单炮 5 万事件级别, `argMax` GROUP BY (shot_no, event_id) 应在 P95 < 1s - 跨炮聚合查询应先按 `is_retracted = 0 AND data_origin = 'REAL'` 过滤 - 频繁 OLAP 场景仍走 ch_events_wide 直接查询(不走 view),只在"当前态准确性"关键场景使用 view

`_version` **来源(必读规则)**: - PG 事务提交后可见的 **outbox_id** 才是 CH 当前态的排序真值 - CH 同步 worker 把 `outbox_id` 作为 `_version` 写入 - `sequence_no` 只保留给过渡期兼容/排障,不再参与当前态决策 - ReplacingMergeTree 自然 merge 不能作为在线"无重复"保证 - `FINAL` 仅用于对账/离线排障,不进入在线路径

若性能不达标的回退路径(V16 明确): - 先实测 `100 / 1000 / 10000` 炮规模下 `ch_events_current_v` 的 P95 - 若 1 万炮跨炮聚合 P95 > 3 s,二期切换到 `AggregatingMergeTree` 当前态派生表

5.2 ch_waveform_preview

```
CREATE TABLE ch_waveform_preview (
  shot_no             UInt64,
  waveform_version_id UInt64,
  channel_id          UInt64,
  channel_code        LowCardinality(String),
  level               LowCardinality(String),
  bucket_start_ns     Int64,
  bucket_width_ns     Int64,
  min_value           Nullable(Float32),
  max_value           Nullable(Float32),
  mean_value          Nullable(Float32),
  rms_value           Nullable(Float32),
  sample_count        UInt32,
  expected_count      UInt32 DEFAULT 0,
  coverage_ratio      Nullable(Float32),
  quality             LowCardinality(String) DEFAULT 'UNKNOWN',
  shot_status         LowCardinality(String) DEFAULT 'UNKNOWN',
  data_completeness   LowCardinality(String) DEFAULT 'UNKNOWN',
  failure_category    LowCardinality(String) DEFAULT 'UNKNOWN',
  data_origin         LowCardinality(String),
  _version            UInt64,
  _delivery_token     String
)
ENGINE = ReplicatedReplacingMergeTree(
  '/clickhouse/tables/{shard}/wf_preview', '{replica}', _version)
PARTITION BY intDiv(shot_no, 10000)
ORDER BY (shot_no, waveform_version_id, channel_id, level, bucket_start_ns)
TTL toDateTime(intDiv(bucket_start_ns, 1000000000)) + INTERVAL 10 YEAR
SETTINGS index_granularity = 8192;
```

缺失桶语义(V19): - P0 阻断规则:

```
sample_count = 0:
  min_value  = NULL
  max_value  = NULL
  mean_value = NULL
  rms_value  = NULL
```

- P1 增强规则:

```
expected_count = 0:
  quality = 'UNKNOWN'
  coverage_ratio = NULL

0 < coverage_ratio < 0.95:
  quality = 'DEGRADED'

coverage_ratio >= 0.95:
  quality = 'GOOD'
```

- 缺失数据绝不能写成 `0`，因为 `0` 是有效物理值

在线查询规则: - API 先查 PG `waveform_versions` 拿 `waveform_version_id` 当前值，再查 `ch_waveform_preview WHERE waveform_version_id = ?` - 不在用户在线路径使用 `FINAL` - 旧 preview 版

本优先靠 `waveform_version_id` 过滤隐藏,批量 TTL/治理任务再清理;同一 key 重投影以最大 `_version` 为准

反范式字段语义(V19 明确): - `shot_status / data_completeness / failure_category` 是 **parse 时刻快照** - PG `shots / shot_failures` 才是当前态真相 - UI 单炮查询可以直接消费 CH 快照字段做下推过滤 - 跨炮统计、月度失败原因排名、正式报表必须以 PG 当前态为准，必要时 `JOIN` PG 或走 reproject - 若 shot 摘要字段事后被修订，必须触发对受影响 `waveform_version_id` 的重投影，不能假设 CH 自动追平

5.3 物化视图

MVP 不启用直接 count() 型 MV(修订/撤销会导致统计漂移)。二期若需 severity 聚合: 1. 从 `ch_events_current_v` 周期性重算 2. 或使用 AggregatingMergeTree 保存 argMax/current-state 派生结果

---

6. Redis 设计(简化)

6.1 用途

```
1. 当前炮状态            : ecrh:shot:current(小键,TTL 24h)
2. 解析进度              : ecrh:parser:job:{job_id}(TTL 1h)
3. 字典缓存              : ecrh:dict:devices / signals(TTL 1h,PG 主动失效)
4. 查询结果缓存          : ecrh:cache:wf:{shot_no}:{wv_id}:{ch}:{level}:{start}:{end}
                           TTL 10 min;waveform_version_id 入 key
5. 短期分布式互斥         : 仅"导出/报告生成"等短任务(TTL ≤ 5 min + Redlock)
                           ⚠️ 长任务(parser)绝不用 Redis 互斥
```

移除: - AOF(不开) - user_sessions(移到 PG `user_sessions` 表) - Parser 长任务锁(迁移到 PG lease)

SLO 条件化: Redis 命中率 / 延迟相关 SLO 仅在"启用 Redis 部署"下生效,不作为 MVP 阻断项。

---

7. 一致性与同步

7.1 写入事务边界

```
BEGIN;
  SELECT nextval('events_event_id_seq') INTO :event_id;
  INSERT INTO event_dedup_keys(shot_no, dedup_key_hash, source_system,
                               source_instance, source_event_key, source_sequence_no,
                               signal_id, event_type, first_event_id)
  VALUES (..., :event_id) ON CONFLICT (shot_no, dedup_key_hash) DO NOTHING
  RETURNING dedup_key_hash;

  INSERT INTO events (event_id, ..., device_code_snapshot, signal_code_snapshot, ...)
  VALUES (:event_id, ...);
  INSERT INTO alarm_state_changes (...);

  INSERT INTO outbox (aggregate_type, aggregate_id, sequence_no,
                      event_type, destination, payload, delivery_token,
                      idempotency_key, status, next_attempt_at, created_at)
  VALUES ('EVENT', :event_id, nextval('outbox_seq'),
          'EVENT_INGESTED', 'CLICKHOUSE', :payload, :delivery_token,
          :idempotency_key, 'PENDING', now(), now());
COMMIT;
```

7.2 Outbox Consumer Pool

(沿用 V14 §7.2 实现伪代码,关键点重申:)

- 多 worker 水平扩展,按 destination 分片
- `FOR UPDATE SKIP LOCKED` 拉取,事务外发送 sink,事务内确认
- worker 拉取、排序、回放一律以 `outbox_id` 为准
- 重试复用同一 `delivery_token`,sink 侧确定性幂等
- 若发现同一 `delivery_token` 跨分区/跨日重新进入,consumer 必须拒绝并写入 DLQ
- 失败 5 次进入 DLQ
- reaper 任务每分钟把 `locked_until < now()` 且仍 `IN_PROGRESS` 的记录置回 `PENDING`

7.3 重放工具

```
ecrh-reproject --scope shot --shot-no 12058 --destination clickhouse
ecrh-reproject --scope waveform --shot-no 12058 --version 2 --destination clickhouse
```

7.4 乱序、迟到、修正

场景	处理
事件乱序	以 event_epoch_ns 为时间真值
迟到	直接入库
事件修正	新事件 `correction_of_(shot_no,event_id)` 指向旧
撤销	写入 `is_retracted=true`,_version 增加
派生层修正	CH 当前态用 `argMax(..., _version)`;PG 是最终真值
重新解析	parser_jobs 触发新 waveform_version + is_current 切换 + reproject

7.5 对账

```
-- 每日 PG vs CH 当前态全字段一致性校验
WITH pg_shot AS (
  SELECT count(*) AS pg_count FROM events
  WHERE shot_no = $1 AND is_retracted = false
)
SELECT pg_count FROM pg_shot;

SELECT count() FROM ch_events_current_v
WHERE shot_no = $1 AND is_retracted = 0;
-- 误差容忍:0
```

每周对账(扩展): - 对象存储 manifest.sha256 ↔ PG waveform_versions.manifest_sha256 - Zarr 通道清单 ↔ waveform_channels 行数 - waveform gap/dropout 汇总 ↔ waveform_channel_gaps 明细 - Outbox PUBLISHED 行数 ↔ CH 行数(按 outbox_id 范围) - **V15 新增**: ch_events_current_v 全字段抽样与 PG 当前态逐字段比较(每周 100 炮抽样) - **V15 新增**: ecrh-raw-tdms / ecrh-zarr 全桶清单 vs PG data_files / waveform_versions 差集 - 误差 → 触发 reprojection_jobs 自动修复

---

8. API 设计

8.1 统一响应 envelope(沿用 V14)

8.2 错误码矩阵(沿用 V14)

8.3 Idempotency-Key

所有 POST 必须接受 `Idempotency-Key` 头,服务端 24h 内同 key 同请求体返回原结果。

8.3A 资源限制与配额

接口/能力	限制
`/events:batch`	≤ 1000 条 / 请求;body ≤ 4 MiB;单条 inline `raw_payload` ≤ 4 KiB
`/shots/{shot_no}/waveforms/{channel}`	level=auto/L3-L5 可整炮;L0/L1 必须限时间窗;max_points 默认 2000,上限 200000;L0 时间窗 ≤ 5 s;L1 ≤ 60 s
`/waveforms:export`	每用户并发 ≤ 2;单任务默认单炮;> 50 GB 必须审批
预签名上传 URL	TTL ≤ 30 min;绑定 object key、part no、checksum、content-length
预签名下载 URL	TTL ≤ 5 min;最小权限;必须审计
admin purge / replay	双人审批;审计链 + 原因字段

8.4 关键端点(V19 增量)

```yaml
paths:
  /events:batch: { post: ... }
  /shots:
    get:
      summary: 按 status / data_completeness / failure_category / retention_class 查询 shot
  /shots/{shot_no}/waveforms/{channel}:
    get:
      summary: 查询单通道波形;支持 `valid_only=true` 仅返回有效窗口
      parameters:
        - { name: valid_only, in: query, schema: { type: boolean } }
  /shots/{shot_no}/waveforms:export: { post: ... }
  /shots/{shot_no}/quality:
    get:
      summary: 返回 shot 级完整性、有效窗口、缺失通道和 parser 质量摘要
  /shots/{shot_no}/failures:
    get:
      summary: 返回失败原因、时间点、严重度、根事件引用
  /export-jobs/{job_id}: { get: ... }
  /parser-jobs/{job_id}: { get: ... }
  /admin/outbox-dlq: { get: ... }
  /admin/outbox-dlq/{outbox_id}:replay: { post: ... }
  /healthz: { get: ... }
  /readyz: { get: ... }
  /admin/files/{file_id}:purge: { post: ... }
  /admin/daq-spool-recovery:
    post:
      summary: 触发 DAQ 本地 spool 扫描与恢复
      x-rbac: [admin, dataops]
      x-approver-required: true

  /admin/reconcile-objects:
    get:
      summary: 列出 object_reconcile_jobs
      parameters:
        - { name: status, in: query }
    post:
      summary: 手动触发一次 reconcile 扫描
      parameters:
        - { name: scope, in: query }
      x-rbac: [admin, dataops]

  /admin/reconcile-objects/{job_id}:
    get: { summary: 查询单个 reconcile job 详情 }
    post:
      summary: 对发现的孤儿对象决策
      parameters:
        - { name: action, in: query, schema: { enum: [REPAIR, QUARANTINE, DELETE] } }
      x-approver-required: true

  /admin/prod-guard/self-test:
    post:
      summary: 注入 8 类模拟数据测试 prod 守卫拦截能力
      x-rbac: [admin, security_admin]
      x-allowed-env: [staging-fake-prod, prod]
      responses:
        '200':
          description: 全部拦截
        '500':
          description: 至少一类未拦截 → 立即 P1

  /data-quality/shots/{shot_no}:
    get:
      summary: 单炮数据质量画像(time_sync / gaps / parser attempts / activation_attempts)
```

```yaml
  /waveform-versions/{shot_no}:
    get:
      summary: 列出某炮所有 waveform_version 历史

  /audit/search:
    get:
      summary: 审计日志检索(按 actor / 资源 / 时间)
      x-rbac: [auditor, admin]
```

`valid_only=true` **规则(V19 明确)**: - 单通道接口优先使用 `waveform_channels.valid_start_ns / valid_end_ns` - 若该通道未写 valid 窗口,回退到 `shots.valid_start_ns / valid_end_ns` - 若两级窗口都为空,回退 raw shot 窗口 `[shot_start_ns, shot_end_ns]`,并返回 `X-Valid-Window: NONE` - 若 raw shot 窗口也为空,返回 `204 NO_CONTENT` + `X-Reason: NO_VALID_WINDOW` - `data_files.valid_*_ns` 仅供恢复流程内部计算,不直接作为 API 响应边界 - `valid_only=true` 只裁剪"已知无效区间",不伪造缺失值

失败事件 → 波形窗口跳转契约(V19 P1):

```
1. UI 调 /shots/{shot_no}/failures 取 root_event_id / failure_time_at
2. API 先查 PG events 或 event_waveform_windows,确定 t_event
3. 默认窗口 = [t_event - 5s, t_event + 5s]
4. 若请求 valid_only=true:
     先与 channel.valid_window 求交集
     交集为空 → 返回空窗口 + reason='OUTSIDE_VALID_WINDOW'
5. L3/L4 可先从 CH 预览返回;需要放大再回源 Zarr L0/L1
```

8.5 复盘 UI(沿用 V14,加强失败炮展示)

P0 最小展示: - `status` - `data_completeness` - `actual_duration_ms` - `duration_s(兼容展示)` - `completion_ratio`

P1 失败炮复盘页: - Shot 编号 / 运行状态 / 数据完整性 - 目标时长 / 实际时长 / 完成比例 - 失败原因 / 严重度 / 根事件 - 有效波形窗口 / 缺失通道数 / 缺失时间段 - retention_class / 是否纳入 normal stats

UI 规则: 1. 缺失区间必须灰色或斜纹显示 2. 不允许把缺失数据画成 `0` 3. normal stats 默认排除 `TRIP / INCIDENT / PARTIAL / RECOVERED_PARTIAL / NO_WAVEFORM` 4. failure analytics 默认包含失败炮和短炮 5. 从失败事件必须能一键跳转到前后 `±5s` 波形窗口

---

9. 可观测性

9.1 分层 SLO(V15 修订)

分阶段门槛:

阶段	L4/L5	L0 首窗	L0 全量
MVP	P95 ≤ 60 min	P95 ≤ 90 min	P95 ≤ 180 min
MVP 试运行	P95 ≤ 30 min	P95 ≤ 45 min	P95 ≤ 90 min
生产试运行	P95 ≤ 30 min	P95 ≤ 45 min	P99 ≤ 120 min

层	SLI	目标	错误预算	告警
L4/L5 总览可见	炮结束 → UI 可看	P95 阶段化	5%/月	超阈值
L0 首窗可查	炮结束 → 任意 L0 小窗	P95 阶段化	5%/月	超阈值
L0 全量可查	炮结束 → 同炮全部 L0	P95/P99 阶段化	5%/月	超阈值
事件入库 P99	≤ 10 s	1%/月	> 30 s	
TDMS 上传成功率	≥ 99.9%	0.1%/月	< 99% 5 min	
单炮 200 GB 上传 P99	≤ 15 min		> 30 min	
TDMS→Zarr 解析 P95	**MVP ≤ 60 min** / 试运行 ≤ 30 min		> 阶段值	
Parser bytes/s(per-worker)	**V16 改为观察性指标**:先连续 4 周实测,再冻结阈值		低于冻结阈值告警	
端到端可复盘(200 GB)	≤ 90 min		> 120 min	
波形 1 s 窗口 P99	≤ 800 ms		> 2 s	
波形整炮总览 P99	≤ 1.5 s		> 5 s	
跨年事件查询 P95	≤ 1 s		> 3 s	
Redis 命中率(若启用)	≥ 90%		< 80%	

层	SLI	目标	错误预算	告警
outbox lag P99	≤ 5 s		> 60 s	
outbox DLQ 计数	= 0		> 0	
PG 复制延迟	≤ 10 s		> 60 s	
CH 同步延迟	≤ 60 s		> 5 min	
prod 出现非 REAL	**永远 = 0**	**0**	**> 0 立即 P1**	
prod_guard_self_test_success	**V15 新增**:每次发布后注入测试 = 1		**失败立即 P1**	
CH visible duplicate count	**V15 新增**: `argMax` 视图与基表非聚合查询差异		持续 > 0 P2	
prod-shadow 一致性	字段分布偏差 < 5%(见下方测量定义)		> 10% P2	
failed_shot_recovery_success_rate	试运行前 ≥ 95%		低于目标告警	
partial_shot_count(7d)	观察性指标		异常抬升告警	
failure_recovery_latency P95	受控中止 ≤ 15 min / spool recovery ≤ 60 min		超阈值告警	
event_to_waveform_jump_success_rate	≥ 99%		< 95% 告警	

9.2 业务质量指标(沿用 V14 + V15 补)

指标	含义
事件 schema reject 率	schema 漂移
time_sync_quality LOW 占比	时钟降级
dropouts_count(每炮总数)	TDMS 完整性
重新解析次数	parser bug 检测
outbox oldest pending age	outbox 堵塞
CH visible duplicate count	当前态查询是否失守
MinIO heal backlog	自愈压力
PG autovacuum lag	表膨胀
CH merge / mutation backlog	CH 压力
object_reconcile_jobs 待处理数	**V15 新增**
parser activation_attempts 累计	**V15 新增**
event_revisions PENDING 待审批	**V15 新增**
`shots.data_completeness != COMPLETE` 占比	失败炮 / 部分炮基线
`valid_only` 查询回退到 shot 窗口次数	channel valid 窗口覆盖不足
recovery manifest 生成失败率	DAQ spool 恢复健康度

prod-shadow 一致性测量定义(V16 新增): - 枚举/类别字段: `chi-square` 比较 `prod-shadow` 与基线分布 - 连续字段: `Wasserstein distance` 或等价分位差度量 - 必须提供每日自动脚本与报告,不能只写抽象阈值

9.3 监控栈

- Prometheus + Grafana(≥ 8 个 Dashboard)
- Loki + correlation_id + shot_no + trace_id
- Alertmanager(分级 + 错误预算燃尽)
- OpenTelemetry trace 贯穿全链
- 必备指标:MinIO PUT/GET P95、Parser bytes/s、PG WAL size、CH parts count

---

10. 安全与合规

10.1 网络分段

(沿用 V14)

10.2 威胁建模(STRIDE,沿用 V14)

10.3 RBAC 角色

- viewer / engineer / physicist / auditor / admin / service / dataops
- admin 分权:data_admin / system_admin / security_admin

10.4 密钥与证书

- **MVP:** 用 `SOPS/age` 或 `systemd-creds` 管理静态密钥材料,禁止裸环境变量长期常驻
- **MVP:** 密钥轮换周期 ≤ 90 天,轮换日志必须可审计
- **MVP:** 对象存储 bucket policy、最小权限 service account、预签名 URL TTL/作用域
- **试运行前:** Vault 集中密钥管理 + 动态凭证
- 证书自动轮换(cert-manager);OT 设备证书 ≤ 1 年
- MinIO SSE-KMS key rotation:每季度
- TLS 1.3;HSTS

10.5 审计

- **MVP:** PG `audit_log` + hash chain + 对象桶 versioning + 服务账号无 DELETE 权限
- **试运行前:** 升级到 Object Lock / WORM
- 所有 admin API 写双份审计:PG + 对象存储

10.6 供应链

- **MVP**: 固定依赖 + 镜像扫描 + 构建留痕 + admin 审计双写 + DB trigger guard + 基础备份
- **试运行前**: SBOM(CycloneDX)+ cosign + admission policy
- 依赖漏洞:HIGH 以上阻断
- IaC 扫描入 CI

10.7 合规

- **MVP**: ADR 级 边界 / 资产 / 数据流 / 保留策略
- **试运行前**: IEC 62443-3-3 + NIST SP 800-82 Rev. 3 对照
- 数据保留:events 10 年、audit 7 年、TDMS 5 年最低

10.8 prod data_origin 三层守卫(V16 硬化)

```
第 1 层 — 应用层(API):
  Ingest Gateway 在 prod 环境拒绝非 REAL data_origin
  失败响应 EVENT_DATA_ORIGIN_FORBIDDEN
  写 audit_log + 触发 SimulatedDataInProduction 告警

第 2 层 — DB trigger(SECURITY DEFINER + system_environment 表):
  trigger 函数 enforce_prod_real_data_origin 由 ecrh_guard_admin 拥有
  从只读 system_environment 表读 env
  prod / prod-shadow 拒绝 NEW.data_origin <> 'REAL'
  业务角色不能修改 system_environment(REVOKE ALL)
  业务角色不能 SET LOCAL 影响 trigger 内的查询

第 3 层 — prod-only CHECK 约束(部署期单独迁移生成):
  ALTER TABLE events ADD CONSTRAINT ck_events_prod_real_only
    CHECK (data_origin = 'REAL') NOT VALID;
  ALTER TABLE events VALIDATE CONSTRAINT ck_events_prod_real_only;
  即使前两层失守,DB 层也无法插入 non-REAL
  dev / staging 不存在该约束

绕过验证(每次发布后):
  POST /admin/prod-guard/self-test
  注入 8 类:
    a) API 注入 SIMULATED → 必须 403 EVENT_DATA_ORIGIN_FORBIDDEN
    b) 普通 service 角色 INSERT events with SIMULATED → 必须 trigger reject
    c) 普通 service 角色尝试 UPDATE system_environment → 必须 ACL deny
    d) 尝试 SET LOCAL ecrh.environment = 'dev' 后 INSERT → trigger 仍 reject(因为不再读 GUC)
    e) 直接 COPY 非 REAL → 必须 CHECK 约束 reject
    f) bulk-import 路由探测 → 必须 404
    g) ecrh-importer 镜像/Pod 探测 → 必须不存在
    h) prod-shadow → prod PG 网络连通性测试 → 必须不可达
  staging 必须维护一套 fake-prod 命名空间,与 prod 使用同一套守卫规则
  PR 阶段 ephemeral env 也必须跑 8 类注入,8/8 通过才能 merge
  任一未拦截 → P1 告警
```

---

11. 容量与硬件规划

11.1 三档场景硬件

MVP / 当前(50ch × 1MHz × 1000s)

角色	规格	数量	说明
DAQ 主机	32C/256G/8×7.68 TB NVMe RAID0/25 GbE	1	本地缓存 4-7 天
App 节点	16C/64G/25 GbE	2	Gateway/Uploader/Normalizer/API/Outbox/监控
Parser Worker	32C/128G/4 TB NVMe	2	可恢复解析
PG Primary/Standby	32C/128G/4 TB NVMe	2	异地副本试运行前补齐
ClickHouse	16C/64G/4 TB NVMe + 12 TB HDD	2	单分片 + 单副本
MinIO	24C/128G/8×7.68 TB NVMe + 12×18 TB HDD	6	按 180 天基线容量修正后上调

收敛后的 MVP 建议规模:约 13-15 台。

近期(100ch × 1MHz × 3000s, 1.2 TB/炮)

- DAQ NVMe 升 16 TB
- 上传链路 25 GbE 起步,实测后升级
- Parser Worker 扩 4-8 个
- CH 扩 3 节点;MinIO 维持 4 节点起步,按 30-90 天热数据扩容

远期典型(~84 TB/炮)

- 必须重做硬件方案(分布式 parser、200 GbE、PB 级对象存储、near-data compute)
- 不在 V19 P0 范围内,作为 §28 触发后的演进

11.2 容量预算(实占,含放大)

年份	MVP 实占	近期实占	远期典型实占
第 1 年(90 天)	~0.35-0.75 PB	~0.6-1.4 PB	~2.7-7.2 PB
第 1 年(180 天,推荐)	~0.70-1.50 PB	~1.2-3.0 PB	~5.4-14.4 PB
第 1 年(365 天上界)	~1.20-2.60 PB	~2.2-6.0 PB	~10.95-29.2 PB

容量运维预警(V19): - 按 180 天基线,6 节点 MinIO 在第 2 年会逼近扩容边界 - 默认把"第 18 个月完成下一轮容量采购 / 扩容决策"设为硬里程碑 - 若 `MinIO used / effective capacity > 70%` 持续 14 天,立即触发扩容流程

---

12. 部署与 CI/CD

12.1 容器化

- OCI 镜像,distroless / python:3.12-slim
- MVP 保留可重现构建 + 镜像扫描;cosign/SBOM 试运行前补齐

12.2 编排

- **dev:** Docker Compose
- **MVP / staging:** 团队已掌握的方式(Compose / systemd / 轻量 K8s),不强制 operator
- **试运行前 / 二期:** 统一到 K8s + CloudNativePG / clickhouse-operator / minio-operator(条件成熟时)

12.3 CI/CD

```
Git Push
    ├── Lint
    ├── Unit test(关键模块 ≥ 85%,总体 ≥ 70%)
    ├── 契约测试(1000 合法 + 1000 非法)
    ├── Integration test(10 GB 假 TDMS)
    ├── DB Migration dry-run
    ├── Prod Guard(部署前):
    │      - prod 部署不含 importer / simulator service
    │      - prod values/config 不含 *.simulated.*
    │      - K8s OPA policy
    ├── 镜像构建 + 漏洞扫描(HIGH 阻断)
    └── 部署到 dev/staging

部署后(V16 强化):
   - 先在 staging fake-prod 自动调用 /admin/prod-guard/self-test
   - 通过后才允许继续放量到 prod
   - 失败 → 自动回滚 + P1 告警

夜间:50 GB 假 TDMS 端到端;Outbox 重放冒烟
每周:200 GB 端到端;kill worker;PG/CH 对账;orphan reconcile 注入
每月:1.2 TB 压测;Chaos / DR 演练
```

12.4 DB Migration

- Flyway(PG)+ 自研 SQL(CH)
- forward-only;rollback 通过新版本兼容
- prod-only CHECK 约束:在独立 prod 迁移包中
- 大表变更:pg_repack;CH 走分区拷贝
- 父分区表索引不得假设“直接并发创建父索引”可用；应按分区并发建索引后再 attach/声明父索引
- 新 CHECK / FK 默认走 `NOT VALID → VALIDATE CONSTRAINT`

12.5 V16/V18 → V19 Schema Migration ADR 摘要(P0 冻结)

目标: - 允许现网从 V16/V17/V18 平滑进入 V19 - 不以热表全表重写为代价换取字段整洁 - 对 `shots/events/outbox/data_files/waveform_channels` 保持业务可写 - 消除 V18 中 generated stored 目标态与 trigger 过渡态混写的问题

核心判断(V19): - `data_completeness / expected_duration_ms / actual_duration_ms / failure_epoch_ns` 可按分区增列 + 分批回填 - `duration_s / include_in_normal_stats` 不再作为分区表 generated stored 目标态，统一由 `v_shots_compat` / `v_shots_normal` 表达 - `shots.failure_category / termination_reason` 删除；失败摘要由 `v_shots_with_latest_failure` 派生 - `event_dedup_keys.event_type` 删除;dedup 契约冲突显式报错 - brownfield 零中断迁移阶段不得引入长期 trigger 兼容列作为新稳态 - 目标 PG 版本固定为 PostgreSQL 16.4+,低于该版本不进入 MVP

推荐阶段:

```
Phase A: 预部署
  1. 为 shots/data_files/waveform_channels/shot_failures/outbox 增加 V19 新列
  2. 新 CHECK/FK 一律 NOT VALID
  3. 新索引按 partition CREATE INDEX CONCURRENTLY
  4. 创建 v_shots_compat / v_shots_with_latest_failure / v_shots_normal

Phase B: 兼容写入
  5. Parser finalize 单点写 shots.data_completeness / actual_duration_ms / completion_ratio
  6. Failure writer 只写 shot_failures,不回写 shots 失败摘要
  7. Event normalizer 切换为 dedup 成功后插入 events
  8. Outbox dispatch SQL 全部切换到 outbox_id

Phase C: 历史回填
  9. 按 shot_no 分区批量回填,每批可中断、可 resume
 10. 回填默认规则:
       COMPLETED → data_completeness=COMPLETE,若 evidence 不足则 UNKNOWN
       ABORTED / INVALID → 先写 UNKNOWN,再按恢复/解析结果二次修订
       expected_duration_ms = expected_duration_s * 1000(若旧字段存在)
       failure_layer = EXPERIMENT(历史物理/工控失败默认)

Phase D: 派生层修复
 11. CH preview 重建为 ReplacingMergeTree
 12. reprojection_jobs 承接受影响 shot / waveform_version / event 的重投影
 13. PG/CH 当前态逐字段对账

Phase E: 校验切换
 14. VALIDATE CONSTRAINT
 15. 验收 PG/CH/对象元数据对账
 16. 零中断验收通过后,移除旧读路径和旧字段
```

生产 DDL 规范: - 大索引:每个子分区 `CREATE INDEX CONCURRENTLY` - 新约束:先 `NOT VALID`,业务稳定后 `VALIDATE` - 禁用 `VACUUM FULL` 作为迁移手段;需要重整时优先 `pg_repack` - 不再把 generated stored 列作为 V19 MVP 目标态

迁移验收(P0 阻断): - 迁移期间业务写入不中断 - 单分区回填可暂停、恢复、重试 - `v_shots_normal` / `v_shots_failures` / `v_shots_with_latest_failure` 结果一致 - 200 GB 端到端 + 失败炮 8 状态矩阵 + PG/CH 对账全部通过 - `shot_failures` FK PoC 在 PG 16.4+ 上通过

---

13. 测试策略## 13. 测试策略

测试	输入规模	验收	频率	阻断
契约	1000 + 1000	schema/error_code 100%	PR	是
单元	业务逻辑	≥ 70%	PR	是
集成	10 GB 假 TDMS	端到端通过	PR	是
上传会话恢复	20/200 GB	断点续传 + 过期 + sha256	周	是
200 GB MVP 端到端	200 GB	闭环	周	是
解析中断恢复	200 GB + kill	无脏 ACTIVE	周	是
Outbox 失败/重放	100 万 + 注入失败	重试复用 token;PUBLISHED/DLQ	周	是
PG/CH 对账	10 炮	误差 = 0	每日	是
波形质量元数据	200 GB	汇总与明细一致	周	是
模拟数据误入生产	8 类注入	全拦截	每发布	是
API 过载防护	超大 batch / window / export	受控错误	周	是
真实 PLC shadow	staging 7-14 天	schema reject = 0	PLC 接入前	是
长脉冲压测	50ch × 1MHz × 10000s	解析 ≤ 60 min	月	远期决策前

测试	输入规模	验收	频率	阻断
Chaos	PG 主备/MinIO/lease	自动恢复	月	生产前
DR	全栈	stretch 2 h / acceptable 4 h;RPO ≤ 5 min	季	生产前
Fuzz	8 类脏行为	不崩溃;DLQ 可解释	PR	是
渗透	OWASP + ICS	无 HIGH	季/上线	是
迁移	近似生产副本	无长锁;兼容	每变更	是
V16/V18→V19 零中断迁移演练	**近似生产副本 + 分区历史数据**	**业务不中断;回填可 resume;约束最终 VALIDATE 通过**	**14.0 前至少 2 次**	**是**
Zarr 年度可读性	抽样 1%	反演 sha256	年	是
prod 直连写非 REAL	**V15 新增**	**必须失败**	**PR**	**是**
CH 当前态全字段一致性	**V15 新增**	**PG/CH 全字段逐字段一致**	**每日**	**是**
orphan object reconcile	**V15 新增**:注入 100 个孤儿	**100% 识别处理**	**周**	**是**
prod-shadow 断网隔离	**V15 新增**	**不可达 prod PG/CH/MinIO**	**每发布**	**是**
SECURITY DEFINER search_path 注入	**V16 新增**	**8 类注入全部 reject**	**周**	**是**
dedup epoch 漂移重传	**V16 新增**	**同 key 不同 epoch 重传 100 次,events 行数 = 1**	**周**	**是**

测试	输入规模	验收	频率	阻断
upload-init 三阶段失败	V16 新增	7 天后无残留 multipart	周	是
activation_attempts 上限	V16 新增	5 次失败后自动 QUARANTINED	周	是
3 秒受控中止炮	V19 P0	`ABORTED + PARTIAL` ;actual_duration_ms 正确;Parser 不补零	PR	是
小于 64 MB 短文件	V19 P0	single PUT 或 1-part multipart;不得补零	PR	是
CH 缺失桶空值语义	V19 P0	`sample_count=0 → min/max/mean/rms = NULL`	PR	是
shot data_completeness 聚合算法	V19 P0	通道级/文件级输入在 8 个状态上结果一致	PR	是
3 秒非受控中断炮	V19 P1	spool recovery 后 `RECOVERED_PARTIAL` 或 `CORRUPT`	试运行前	是
只有事件无波形	V19 P1	`NO_WAVEFORM` ;事件可查,波形查询返回空窗口	试运行前	是
TDMS 文件损坏	V19 P1	`CORRUPT` ;Parser `QUARANTINED`	周	是
失败炮复盘 UI	V19 P1	显示实际时长、完成比例、有效窗口、缺失区间	MVP 试运行	是
生产试运行失败回退	V16 修订	stretch 2 h / acceptable 4 h	生产试运行前至少 2 次	是

14. 分阶段实施路线(V19 修订)

14.0 V19 P0 / MVP 阻断项(2-3 周)

```
目标:先把短炮 / 失败炮的最小真实语义补到可上线
要做:
  - `shots.data_completeness` + `actual_duration_ms` 落地
  - `expected_duration_ms` 统一毫秒口径
  - `duration_s` 改为由 `actual_duration_ms` 生成的兼容字段
  - `shot_failures` MVP 最小集落地
  - `failure_epoch_ns` 前移到 P0
  - Parser 明确禁止补零,按实际 `actual_samples` 生成 Zarr
  - shot 级 `data_completeness` 聚合规则落地并单测
  - `< 64 MB` 文件 single PUT / 1-part multipart 规则明文化
  - `ch_waveform_preview` 缺失桶 `sample_count=0 → 聚合值 NULL`
  - `events.message / outbox.payload` 体积上限落地
  - `V16/V18→V19 schema migration ADR` + dev 演练通过
交付:DDL + 迁移 ADR + 最小解析规则说明
退出:3 秒受控中止炮用例通过;短文件用例通过;缺失桶语义通过;零中断迁移演练通过
```

14.1 MVP 开发(3-4 月)

```
目标:跑通当前 200 GB/炮规模工程闭环
要做:
  - PG 事实源(含 system_environment、object_reconcile_jobs、findings 子表、最小主数据表)
  - TDMS multipart 上传 + reconcile_status 闭环
  - Parser 可恢复 + activation_attempts 追踪
  - `shots.status × data_completeness` 双维查询打通
  - Zarr L0 + L4/L5
  - CH 事件宽表 + L3/L4/L5 预览 + 全字段 argMax 当前态视图
  - Outbox v2 + 分区 + 归档 + outbox_id 版本语义
  - data_origin 三层守卫
  - mTLS / OIDC / RBAC 基础版
  - 最小监控与备份
  - 最小安全门槛:bucket versioning / 服务账号无 DELETE / admin 审计双写
  - prod-guard self-test 自动化(fake-prod 周跑)
交付:上传→解析→事件→波形→UI 闭环
退出:200 GB 端到端通过;kill worker 无脏 ACTIVE;PG/CH 全字段对账 0;orphan 100% 识别;prod-guard
8 类全拦截
```

14.2 V19 P1 / 试运行前闭环(1-2 月)

```
目标:把“能跑”补到“能真实复盘失败炮”
要做:
  - DAQ spool recovery
  - 5 个 recovery 组件逐个补齐输入/输出契约、幂等、SLO、Runbook
  - recovery manifest / recovery_note / recovery_tool_version
  - `coverage_ratio` 物化与 partial channel 统计
  - failure analytics 页面
  - Replay UI / Browse UI / 报告闭环
  - 失败炮复盘页
  - 保护事件与时间质量展示(LOW 强红色)
  - `activation_attempts > 5 → QUARANTINED` 在文档、DDL、测试、监控四处一致
  - `time_sync_quality LOW` 传播规则落地
  - 失败事件 → 波形窗口 `±5s` 跳转
  - `valid_only=true` 通道窗口 / shot 窗口回退规则落地
  - 失败炮专属 SLI 与 Dashboard
  - Runbook / 告警 / on-call
  - staging / prod-shadow 真实 PLC shadow
交付:工程师可“找到失败原因 → 看有效波形 → 形成复盘报告”
退出:L4/L5 ≤ 30 min;L0 首窗 ≤ 45 min;L0 全量 ≤ 90 min;失败炮专项用例通过
```

14.3 生产试运行准入(2 月)

```
目标:把"可复盘 MVP"提升为"可申请生产试运行"
要做:
  - prod-shadow 14-30 天真实 PLC shadow(0 网络通路 prod 事实库)
  - 若资源评估不足,允许降到 7-14 天,但必须有书面风险签署
  - 备份恢复演练
  - DR 演练
  - 审计/证据 Object Lock + WORM
  - Vault 集中密钥
  - SBOM / 镜像签名 / 安全评审
  - 失败回退演练(§14.7,至少 2 次)
交付:生产试运行准入报告
退出:schema reject = 0;影子对比达标;恢复/安全/回退演练通过
```

14.4 二期建设(3-6 月)

```
目标:完善平台工程化
要做:
  - 视压测决定 L2 近 30 天进 CH
  - 自动根因推断
  - 多系统失败关联图
  - 跨 campaign 失败模式挖掘
  - 完整 ISA-18.2 报警生命周期
  - HDF5 关键炮镜像(关键 100% + 普通 1%)
  - K8s/operator 规范化
  - 对象存储冷热分层
  - calibration_versions 表落地
交付:稳定试运行的完整平台基线
退出:prod-shadow 14-30 天稳定;schema reject 低于阈值
```

14.5 远期演进(6-12 月)

```
目标:面向长脉冲、高采样、PB 级
要做:
  - Temporal / 分布式 Parser
  - PB 级对象存储
  - near-data compute
  - 200 GbE / InfiniBand
  - 1 TB / 10 TB 专项压测
交付:演进 ADR + 采购输入
退出:专项压测通过后采购
```

14.6 阶段间退出门槛汇总

阶段	关键退出
14.0 → 14.1	V19 P0 关闭;DDL 冻结;迁移 ADR 演练通过
14.1 → 14.2	200 GB 端到端;PG/CH 全字段对账 0;prod-guard 8/8 拦截;orphan 100% 识别
14.2 → 14.3	L4/L5 ≤ 30 min;L0 首窗 ≤ 45 min;真实 PLC shadow 7 天 schema reject = 0
14.3 → 生产试运行	14-30 天 shadow;DR;WORM;失败回退 stretch 2 h / acceptable 4 h;安全评审通过

14.7 生产试运行失败回退 Runbook(V16 修订)

```
触发条件(任一):
  - prod 出现非 REAL 数据(SLO 红线突破)
  - prod 入口 schema reject 率 > 1%
  - 真实 PLC 接入后 1 h 内 P1 告警 ≥ 3 次
  - prod-shadow 与 prod 字段分布偏差 > 20%
  - 安全自检 prod-guard self-test 失败

决策流程(15 min 内):
  1. on-call 工程师 + 值班 SRE 确认告警真实性
  2. 通知 incident commander + 安全负责人 + 项目负责人
  3. 启动失败回退会议,决定:
     a) 临时降级(限流 / 熔断 specific source_instance)
     b) 完全回退到 prod-shadow

完全回退执行步骤(目标:stretch 2 h / acceptable 4 h):
  T+0 min  : 停止真实 PLC 推送 prod
             - 在 Ingest Gateway 注入拒绝规则:source_system='REAL_PLC' → 503
             - PLC 端本地 SQLite WAL 缓冲自动接管,不依赖远程通知
  T+5 min  : 冻结 prod 数据写入
             - prod 库 events / shots 设置 ALTER TABLE ... DISABLE 写入(临时 trigger)
             - 标记本次试运行窗口为 INCIDENT
  T+10 min : 切换流量到 prod-shadow
             - client-side circuit breaker / Service Mesh 切到 prod-shadow 命名空间
             - prod-shadow 重新启动接收真实流量(此前为镜像模式)
  T+20 min : 数据完整性核对
             - 对比 prod 最后 1 h 写入与 prod-shadow 镜像数据
             - 标识需事后修复的差集 → 写入 incident report
  T+30 min : 出具事故快报
             - 影响时间窗、影响数据范围、回退步骤、根因初判
             - 通知所有客户端
  T+120 min : 目标完成回退
             - prod 进入 read-only 模式,等待修复
             - 建立 incident bridge 直到根因消除

回退后处理:
  [ ] 24 h 内出根因分析报告
  [ ] 数据修复方案(prod incident 期数据 → prod-shadow / 隔离桶)
  [ ] 回退演习经验写入 runbook
  [ ] 决定是否再次申请生产试运行(至少 7 天冷却期)

演练要求:
  - 试运行准入前必须至少演练 2 次完整失败回退
  - stretch 2 h / acceptable 4 h 必须实测
  - 每年至少演练 1 次
```

---

15. 风险与应对

风险	概率	影响	应对
远期容量低估	中	极高	§0.2 三档表 + §28 触发条件
event_dedup_keys 与 events 写入冲突	低	极高	§4.2 显式序列 + 同事务
Parser activation 指针不一致	中	极高	§2.2 immutable build + activation_attempts
Outbox 失败重放可见重复	中	高	§7.2 delivery_token + sink 幂等
模拟数据污染 prod	低	极高	§10.8 三层守卫 + self-test
Zarr 长期格式衰退	低	高	§3.2 年度健康报告 + HDF5 镜像
TDMS 过早删除	中	高	MVP/试运行不自动删除
时钟漂移首出错判	中	高	time_sync_quality + UI confidence
schema 漂移	高	高	契约先行 + ADR + CI
brownfield schema 迁移锁表/重写	中	极高	§12.5 迁移 ADR + 分区回填 + NOT VALID/VALIDATE + dev 演练
真实 PLC 切换翻车	中	极高	§23 阶段 A→D + §14.7 失败回退
MVP 范围失控	高	中	§14 分阶段 + 立即修正阶段
对象孤儿堆积	中	中	object_reconcile_jobs + 周扫描
prod 试运行翻车	低	极高	§14.7 stretch 2 h / acceptable 4 h

16. 上线前检查清单(V19 修订)

```
── 架构与 P0 ──
[ ] events 分区唯一索引 UNIQUE(shot_no, dedup_key_hash) 已建
[ ] events_event_id_seq 预分配路径已实测;event_dedup_keys / events 同事务闭环
[ ] event_dedup_keys hash 公式 ADR 通过 + 入库 SQL 单测通过
[ ] Parser PG lease + fencing token 实测:lease 过期不丢数据
[ ] Parser activation 已改为 immutable build + manifest activation;activation_attempts 可观
测
[ ] Outbox 状态机完整;PUBLISHED 30-90 天分区归档脚本演练
[ ] outbox / delivery_ledger / audit_log 月分区已自动建
[ ] CH `_version` 已改为 outbox_id;并发修订压测通过
[ ] 在线查询路径不依赖 FINAL;ch_events_current_v 全字段 argMax 已验证
[ ] upload_sessions 已含 storage_upload_id / session_expires_at / completed_parts_sha256 /
reconcile_status
[ ] ck_init_phases 已落地;upload-init 三阶段失败注入通过
[ ] UNIQUE(client_id, idempotency_key) 已落地
[ ] waveform_channels / waveform_channel_gaps 已落库并对账
[ ] active_experiment_days_per_year 三档容量表已签字
[ ] system_environment 表已部署 + SECURITY DEFINER 函数 search_path 硬化 + 最小特权 owner
[ ] prod-only CHECK 约束已 VALIDATE 且 self-test 可探测漂移
[ ] object_reconcile_jobs / object_reconcile_findings 已建 + 注入 100 孤儿全识别
[ ] dedup hash 已删除 event_epoch_ns;同 key 不同 epoch 重传测试通过
[ ] `expected_duration_ms / actual_duration_ms` 毫秒口径已统一
[ ] `shots.data_completeness` 聚合算法单测通过
[ ] `events.message / outbox.payload / outbox_dlq.payload` 体积上限已落地
[ ] `V16/V18→V19 schema migration ADR` 已演练 2 次且可 resume

── 系统功能 ──
[ ] 全部 P0 监控指标接入 + 演练告警
[ ] PG PITR + 跨区副本 + 周演练
[ ] CH 备份恢复演练
[ ] DAQ → 上传 → Zarr → CH 端到端达标
[ ] 200 GB 端到端压测通过
[ ] 200 GB + kill worker 无脏 ACTIVE
[ ] 50ch × 1MHz × 1000s 持续 24h 不丢帧
[ ] RBAC 角色 + audit_log 演练
[ ] DB Migration forward + 兼容
[ ] brownfield 迁移期间业务零中断验收通过
[ ] Runbook ≥ 13 类(含 §14.7 失败回退)
[ ] staging / prod-shadow 真实 PLC shadow 通过
[ ] DR RTO stretch 2 h / acceptable 4 h;RPO ≤ 5 min 演练
[ ] 生产试运行失败回退演练至少 2 次,实测 acceptable < 4 h

── 数据来源(含模拟)──
[ ] contracts/event.schema.json 与 PLC 工程师签字
[ ] events / shots / CH / 导出 全含 data_origin
[ ] UI 在所有非 REAL 数据显示水印
[ ] cleanup_simulated.sql 演练
[ ] prod 关闭 importer endpoint
[ ] prod data_origin 三层守卫齐备(API + trigger + CHECK)
[ ] prod-guard self-test 8 类全拦截 + 自动跑在 staging fake-prod 与每次发布后
[ ] prod-shadow 与 prod 事实库 0 网络通路(NetworkPolicy + Firewall 双重)
[ ] 阶段 A→D 切换 SOP 完成

── 安全 ──
[ ] STRIDE 威胁模型 ADR
[ ] mTLS 端到端
[ ] 对象存储 bucket policy / 预签名 URL TTL / 作用域演练
[ ] MVP:evidence/audit bucket versioning + 服务账号无 DELETE 权限演练
[ ] 试运行前:Object Lock / WORM 演练
[ ] 若 K8s:cosign + admission policy
[ ] 集中密钥 → 动态凭证轮换演练

── 文档 ──
```

```
[ ] 架构、API、数据模型、运维、应急、Runbook、ADR ≥ 13 篇
[ ] V19 新增 / 修订 ADR:
    - system_environment / readiness gate
    - prod data_origin guard
    - CH current-state argMax view
    - event dedup hash formula
    - object reconcile workflow
    - outbox partition strategy
    - V16/V18→V19 schema migration
    - production rollback runbook
```

---

17. 关键交付物清单

```
1.  V19 完整设计文档(本文档)
2.  PostgreSQL 完整 DDL(含 system_environment、最小主数据表、events_event_id_seq、shot-scoped
dedup、
    upload_sessions、object_reconcile_jobs/findings、outbox 分区、所有 V19 字段)
3.  ClickHouse 完整 DDL + ch_events_current_v 全字段 argMax view + `_version=outbox_id`
4.  Zarr v3 manifest schema + 年度健康报告模板 + reader_min_version 格式约束
5.  OpenAPI 3.1 完整 YAML(含 fake-prod self-test 约束)
6.  contracts/event.schema.json + event.proto v1
7.  TDMS Parser 工作流参考实现(PG lease + checkpoint + activation_attempts + reaper)
8.  Outbox v2 + Consumer Pool + reprojection 工具
9.  ecrh-simulator 项目骨架(8 类脏行为)
10. event_importer CLI(仅 dev/staging)
11. cleanup_simulated.sql + CH 同步清理
12. 部署资产 + prod 守卫(三层)+ prod-guard self-test 注入器
13. object_reconcile_jobs 扫描器 + 处置 CLI
14. 监控 Dashboard JSON × 8 + 错误预算燃尽
15. 告警规则 YAML(SimulatedDataInProduction / shadow 一致性 / prod_guard_self_test_failure)
16. Runbook ≥ 13 SOP(含失败回退)
17. 真实 PLC 切换 SOP(阶段 A→D)
18. 历史回放 + 对账修复工具
19. 200 GB MVP 验证报告(含 prod-guard 8 类、orphan 注入、CH 全字段对账、并发修订 version 语义)
20. 安全配置基线
21. 上线检查清单(V19)
22. ADR 集 ≥ 13 篇:
    - 容量口径
    - CH 当前态查询
    - Outbox 生命周期
    - Parser 恢复
    - V16/V18→V19 schema migration(V19 新增)
    - prod data_origin guard(V16 修订)
    - event dedup hash formula(V16 修订)
    - object reconcile workflow(V16 修订)
    - production rollback runbook(V16 修订)
23. V16/V18→V19 migration 演练报告(含分区回填、零中断验收、索引/约束 VALIDATE 记录)
```

第二部分:模拟数据驱动开发的工程纪律(沿用 V14 §18-§26 主体)

---

18. 数据来源四类型

取值	含义	允许环境
`REAL`	真实 PLC/HMI/SCADA 推送	dev / staging / prod-shadow / prod
`SIMULATED`	人工模拟	dev / staging
`SYNTHETIC`	程序合成	dev / staging
`REPLAYED`	历史回放	dev / staging

prod / prod-shadow 仅允许 REAL;其他被 API + DB trigger + CHECK 三重拒绝并写 audit。

V19 prod/prod-shadow dataset CHECK:

```sql
-- 仅 prod-shadow 库存在
ALTER TABLE shots ADD CONSTRAINT ck_shadow_dataset
CHECK (dataset_id LIKE 'shadow-%') NOT VALID;
ALTER TABLE events ADD CONSTRAINT ck_events_shadow_dataset
CHECK (dataset_id LIKE 'shadow-%') NOT VALID;
ALTER TABLE data_files ADD CONSTRAINT ck_data_files_shadow_dataset
CHECK (dataset_id LIKE 'shadow-%') NOT VALID;

-- 仅 prod 库存在
ALTER TABLE shots ADD CONSTRAINT ck_prod_not_shadow_dataset
CHECK (dataset_id IS NULL OR dataset_id NOT LIKE 'shadow-%') NOT VALID;
ALTER TABLE events ADD CONSTRAINT ck_events_prod_not_shadow_dataset
CHECK (dataset_id IS NULL OR dataset_id NOT LIKE 'shadow-%') NOT VALID;
ALTER TABLE data_files ADD CONSTRAINT ck_data_files_prod_not_shadow_dataset
CHECK (dataset_id IS NULL OR dataset_id NOT LIKE 'shadow-%') NOT VALID;
```

---

19. 10 条强制工程纪律(沿用 V14)

19.1-19.5 保留。

19.6【D-6】Importer 仅 dev/staging(V15 强化)

```
prod / prod-shadow 关闭模拟数据导入路径,六层防御:
1. API Gateway:路由 /v1/internal/events:bulk-import 在 prod 部署不存在
2. 部署层:prod 不部署 importer service
3. OPA Gatekeeper(K8s):拒绝 prod 命名空间的 ecrh-importer 镜像 / *.simulated.* configmap
4. Network Policy:prod-shadow 与 prod 事实库 PG/CH 0 网络通路
5. PG SECURITY DEFINER trigger guard:prod 拒绝 data_origin != 'REAL'
6. PG prod-only CHECK constraint:DB 层兜底
7. Prometheus alert: SimulatedDataInProduction
8. /admin/prod-guard/self-test 每发布后自动验证
```

19.7-19.10 保留。

---

20. 模拟器项目结构(沿用 V14)

---

21. 事件契约(沿用 V14,加 schema_version)

---

22. 8 类脏行为注入清单(沿用 V14)

---

23. 真实 PLC 切换 SOP(沿用 V14)

(与 V14 §23 完全一致;失败回退见 §14.7)

---

24. 环境隔离矩阵(沿用 V14)

---

25. Importer CLI 工具规范(沿用 V14)

---

26. 验收清单(沿用 V14 + V15 增量)

```
── prod-shadow 隔离 ──
[ ] prod-shadow 命名空间网络策略:与 prod 事实库 PG/CH/MinIO 0 连通
[ ] prod-shadow data_origin 全部 REAL,但 dataset_id 前缀 'shadow-'
[ ] prod-shadow 报告生成
[ ] 退役流程:30 天数据保留窗口

── V16 / V17 增量 ──
[ ] system_environment 表初始化值与实际部署环境一致
[ ] staging fake-prod 与 prod 发布后都跑 /admin/prod-guard/self-test,8 类全拦截
[ ] object_reconcile_jobs 注入 100 孤儿,100% 识别
[ ] PG/CH 全字段一致性对账(ch_events_current_v vs events)
[ ] 失败回退演练 stretch 2 h / acceptable 4 h
[ ] 3 秒受控中止炮 → `ABORTED + PARTIAL`
[ ] `duration_s` 自动派生自 `actual_duration_ms`
[ ] `sample_count=0` 的 CH 缺失桶聚合值为 `NULL`
```

---

27. 演进触发条件(沿用 V14)

组件	触发(任一)	评估
Kafka / Redpanda	事件峰值 > 5 k/s 持续 / 异步消费者 ≥ 3 类 / 跨数据中心	ADR:Kafka 替换/并行 outbox
Debezium CDC	PG→CH 派生表 ≥ 5 个 / 任意时点回放	替换 outbox 轮询
Temporal/Argo	parser 平均时长 > 30 min / 取消/分支/补偿	迁移 parser 工作流
Lakehouse(Iceberg)	10 年归档 / 跨装置共享 / 审计血缘	二期 PoC
TimescaleDB	不计划用作主波形库	否决
OpenSearch	日志全文搜索 / 事件 message 检索	单独副本
远期硬件	单炮 > 10 TB / 持续 > 5 GB/s	触发 §11.1 远期方案

---

28. 一句话总结

> V19 是在 V18 完整架构基线上形成的 MVP 冻结版：保留失败炮模型、Outbox、Parser fencing、prod guard、对象存储/Zarr/PG/CH 分层等主体设计，同时关闭 V18 评审指出的 P0/P1 阻断项，包括 Migration ADR 目标态、shot 聚合算法、events dedup 顺序、CH preview version 语义、shots 失败摘要双写、outbox dispatch 顺序、prod-shadow CHECK、multipart abort 与 reprojection_jobs MVP 化。V19 可作为 14.1 MVP 实施输入。